UNIVERSIDAD DE SANTIAGO DE COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Implementación software y hardware de un algoritmo de criptografía ligera

Autor:
Darío Aldrey Muiño

Tutores:
David López Vilariño
María Purificación Cariñena Amigo

Grado en Ingeniería Informática

Julio 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de Ingeniería de la Universidad de Santiago de Compostela para la obtención del Grado en Ingeniería Informática

Acrónimos

- SPN: Substitution–Permutation Network
- HLS: High-Level Synthesis
- FPGA: Field-Programmable Gate Array
- LUT: Look-Up Table
- ARM: Advanced RISC Machine
- NIST: National Institute of Standards and Technology
- HDL: Hardware Description Language
- SoC: System on Chip
- PS: Processing System
- PL: Programmable Logic
- FF: Flip-Flop
- BRAM: Block RAM
- LUTRAM: LUT RAM (RAM implementada en LUTs)
- IP: Intellectual Property (IP core)
- ECB: Electronic Codebook
- CTR: Counter Mode
- AXI: Advanced eXtensible Interface
- FIFO: First-In First-Out
- PYNQ: Python Productivity for Zynq

Resumen

El incremento de dispositivos IoT y sistemas embebidos ha hecho necesaria la utilización de algoritmos de criptografía ligera capaces de proporcionar mecanismos de seguridad con un bajo consumo de recursos. En este contexto, los cifrados basados en redes de sustitución y permutación (SPN, del inglés Sustitution Permutation Network) constituyen una alternativa ampliamente empleada en entornos con limitaciones de potencia, memoria y capacidad de procesamiento.

En este trabajo se realiza el análisis, diseño e implementación de un algoritmo de criptografía ligera de tipo SPN sobre la plataforma Pynq-Z2, basada en un System on Chip (SoC) que integra un procesador ARM y lógica programable FPGA. Se desarrollaron implementaciones software en Python e implementaciones hardware mediante síntesis de alto nivel (HLS, High Level Synthesis), permitiendo comparar ambas aproximaciones en términos de rendimiento, latencia, uso de recursos y consumo de energía.

Como caso de estudio inicial se implementó una versión simplificada de cifrado SPN de 32/64 bits, sobre la que se aplicaron distintas técnicas de optimización tanto software como hardware. Posteriormente, la metodología se extendió al algoritmo PRESENT-80, utilizado habitualmente en el ámbito de la criptografía ligera.

Las implementaciones software se desarrollaron en Python y se ejecutaron sobre el procesador ARM de la plataforma mediante el entorno Jupyter proporcionado por Pynq. Por otra parte, las implementaciones hardware siguieron un flujo de trabajo basado en síntesis de alto nivel (HLS), empleando Vitis HLS para la generación de bloques IP e integrándolos posteriormente mediante Vivado en el sistema junto con los buses AXI y el procesador ARM. Finalmente, se generó el bitstream correspondiente y se desplegó sobre la FPGA de la plataforma.

Los resultados obtenidos muestran una mejora significativa de las versiones hardware frente a las software en términos de latencia y throughput, así como el impacto de las distintas optimizaciones sobre el uso de recursos y el rendimiento global del sistema.

Como resultado del trabajo, se han obtenido bloques IP reutilizables capaces de ser integrados en función de los requisitos del sistema dentro de un flujo de diseño basado en Vivado. Esta modularidad facilita su reutilización en distintos escenarios de aplicación y su despliegue en entornos embebidos reales.

Finalmente, se desarrolló una demostración práctica orientada al cifrado de datos reales, validando la aplicabilidad de la solución en escenarios embebidos y sistemas de procesamiento seguro.

Este TFG corresponde a la modalidad Tipo A.

Índice general

1. Introducción **1**
1.1. Criptografía ligera 1
1.2. Redes de sustitución-permutación (SPN) 2
1.3. Plataforma FPGA y entorno de desarrollo 3
1.4. Problema abordado y métricas de evaluación 4
1.5. Hipótesis y objetivos 5

2. Estado de conocimiento del problema a abordar **7**
2.1. Criptografía ligera y PRESENT 7
2.2. Técnicas de optimización relevantes 8
2.3. Síntesis HLS y plataformas heterogéneas 8
2.4. Diferencias con el presente trabajo 9

3. Metodología **11**
3.1. Materiales 11
3.1.1. Plataforma hardware 11
3.1.2. Herramientas de desarrollo 12
3.1.3. Entorno experimental 12
3.2. Implementaciones Software 12
3.2.1. Implementación básica 12
3.2.2. Implementaciones basadas en tablas de búsqueda 13
3.2.3. Implementación basada en *bit-slicing* 14
3.3. Implementaciones Hardware 16
3.3.1. Flujo de implementación hardware 16
3.3.2. Implementacion básica 17
3.3.3. Implementaciones basadas en *loop unrolling* 17
3.3.4. Implementaciones basadas en paralelización 18
3.3.5. Implementaciones basadas en segmentación (*pipelining*) . . 19
3.3.6. Implementación basada en serialización 20
3.4. PRESENT-80 21

4. Pruebas **23**

5. Discusión de los resultados **25**
5.1. Métricas . . . 25
5.1.1. Latencia . . . 26
5.1.2. Throughput . . . 26
5.1.3. Consumo de memoria . . . 26
5.1.4. Área ocupada . . . 27
5.1.5. Consumo de potencia . . . 27
5.2. Resultados de las métricas software . . . 28
5.2.1. Latencia . . . 28
5.2.2. Rendimiento o Throughput . . . 30
5.2.3. Consumo de memoria . . . 31
5.3. Resultados de las métricas hardware . . . 32
5.3.1. Latencia . . . 32
5.3.2. Rendimiento o Throughput . . . 34
5.3.3. Consumo de Área . . . 36
5.3.4. Consumo de potencia . . . 39
5.4. Métricas para el PRESENT-80 . . . 40
5.5. Modo de cifrado CTR . . . 42

6. Conclusiones y posibles ampliaciones **45**

A. Manuais técnicos **47**
A.1. Material proporcionado . . . 47
A.2. Preparación del entorno PYNQ . . . 48
A.2.1. Grabación de la imagen en la tarjeta microSD . . . 48
A.2.2. Primer arranque de la placa . . . 48
A.2.3. Acceso al entorno PYNQ . . . 49
A.2.4. Acceso al sistema de archivos . . . 50
A.3. Preparación del entorno Vivado y Vitis . . . 50
A.3.1. Requisitos y versión utilizada . . . 50
A.3.2. Vitis HLS: generación e importación de IP . . . 50
A.3.3. Vivado: integración del sistema y generación del bitstream 51
A.4. Reproducción de las métricas . . . 51

Bibliografía **53**

Índice de figuras

1.1. Ejemplo de estructura SPN [1] 3

3.1. Ejemplo de diseño paralelo [1] 19
3.2. Ejemplo de diseño pipeline [1] 20
3.3. Ejemplo de diseño serializado [1] 21

5.1. Comparativa de mapas de utilización 37
5.2. Comparación entre ECB y CTR 43
5.3. Caso práctico de cifrado de imagenes CCTV [13] 43

A.1. Guía de arranque de pynq-z2 49

Índice de tablas

3.1. Especificaciones de la placa PYNQ-Z2 [14]. 11
3.2. Mapeo de la S-box de 4 bits de PRESENT 12
3.3. Tabla de permutación para los 4 bits menos significativos 14

4.1. Vectores de prueba para SPN (en formato hexadecimal). 23
4.2. Vectores de prueba para PRESENT-80 (en formato hexadecimal). 24

5.1. Comparación de latencia media para implementaciones software . 29
5.2. Comparación de rendimiento para implementaciones software . . . 30
5.3. Comparación de consumo de memoria para implementaciones software 32
5.4. Comparación de latencia media en microsegundos (μs) para implementaciones hardware 34
5.5. Comparación de throughput ajustado en miles de bloques por segundo (kbloques/s) para implementaciones hardware 35
5.6. Comparación de consumo de área del diseño completo en la FPGA 39
5.7. Desglose de potencia dinámica en la lógica programable (PL) para las diferentes implementaciones hardware 40
5.8. Métricas finales de la implementación PRESENT en SW. 41
5.9. Métricas finales de la implementación PRESENT en HW. 42

Capítulo 1

Introducción

La creciente proliferación de dispositivos pertenecientes al Internet de las Cosas (IoT) y de sistemas embebidos ha impulsado la necesidad de incorporar mecanismos de seguridad capaces de proteger la información procesada y transmitida por estos dispositivos. Sensores inalámbricos, dispositivos médicos, sistemas de monitorización industrial, cámaras inteligentes o dispositivos *wearables* constituyen algunos ejemplos de entornos donde la seguridad resulta fundamental debido al intercambio continuo de datos potencialmente sensibles.

Sin embargo, este tipo de dispositivos suele estar sujeto a importantes restricciones de memoria, capacidad de procesamiento y consumo energético, lo que dificulta la utilización de algoritmos criptográficos convencionales diseñados para sistemas con mayores recursos computacionales. Aunque algoritmos ampliamente utilizados como AES proporcionan elevados niveles de seguridad, su implementación puede resultar costosa en determinados entornos embebidos con limitaciones estrictas de área hardware, consumo de energía o rendimiento computacional.

Ante esta situación surge la criptografía ligera, cuyo objetivo consiste en desarrollar mecanismos criptográficos capaces de proporcionar un nivel de seguridad adecuado minimizando el consumo de recursos hardware y software. Dentro de este ámbito, los cifrados basados en redes de sustitución y permutación (SPN) han adquirido una gran relevancia debido a su simplicidad estructural y su capacidad de adaptación a plataformas con recursos limitados. Un ejemplo representativo es PRESENT, un cifrado en bloque diseñado específicamente para entornos con fuertes restricciones de recursos hardware y energía [2].

1.1. Criptografía ligera

La criptografía ligera engloba un conjunto de algoritmos y técnicas diseñados específicamente para dispositivos con recursos limitados, donde restricciones de hardware, memoria, capacidad de procesamiento y consumo energético condicionan el diseño de los mecanismos de seguridad. En estos entornos, resulta necesario

alcanzar un equilibrio entre el nivel de seguridad ofrecido y el coste asociado a su implementación.

Dependiendo de la aplicación objetivo, los algoritmos criptográficos pueden orientarse hacia implementaciones software o hardware. Las implementaciones software suelen ofrecer una mayor flexibilidad, facilidad de actualización y menor coste de desarrollo, mientras que las implementaciones hardware permiten explotar el paralelismo inherente de determinados algoritmos, alcanzando mayores prestaciones en términos de latencia y rendimiento. En este contexto, las implementaciones sobre lógica programable representan una solución intermedia, al proporcionar la flexibilidad característica del software junto con prestaciones próximas a las del hardware dedicado.

1.2. Redes de sustitución-permutación (SPN)

Una red de sustitución–permutación (SPN, Substitution–Permutation Network) es un tipo de cifrado en bloque compuesto por múltiples rondas sucesivas en las que se aplican una serie de transformaciones sobre el estado interno de los datos. Este tipo de arquitectura constituye la base de numerosos algoritmos criptográficos modernos debido a su capacidad para proporcionar seguridad mediante estructuras relativamente simples y eficientes. La estructura general de una SPN se muestra en la Figura 1.1.

Cada ronda de una SPN suele estar formada por tres etapas principales:

- **Sustitución.** Consiste en reemplazar pequeñas porciones del estado interno, habitualmente grupos de bits o nibbles, mediante funciones no lineales implementadas a través de S-boxes (Substitution boxes). Esta operación introduce confusión en el cifrado, dificultando la relación entre la clave secreta y el texto cifrado generado.

- **Permutación.** Redistribuye los bits del estado interno con el objetivo de propagar la influencia de cada bit a lo largo de todo el bloque de datos. Esta etapa aporta difusión al algoritmo, haciendo que pequeñas modificaciones en la entrada produzcan cambios significativos en la salida.

- **Adición de clave.** Incorpora la clave secreta al proceso de cifrado, normalmente mediante operaciones XOR entre el estado interno y subclaves derivadas de la clave principal.

La repetición de estas operaciones a lo largo de múltiples rondas permite alcanzar los principios de confusión y difusión definidos por Claude Shannon, fundamentales en el diseño de sistemas criptográficos seguros [3].

En el ámbito de la criptografía ligera, las arquitecturas SPN resultan especialmente adecuadas debido a la simplicidad de sus operaciones internas y a su bajo

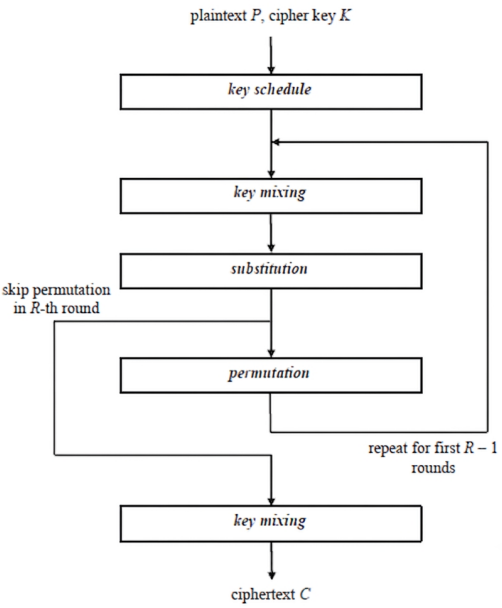


Figura 1.1: Ejemplo de estructura SPN [1]

coste de implementación. Parámetros como el tamaño de bloque, la longitud de clave o el número de rondas se ajustan con el objetivo de equilibrar seguridad y consumo de recursos. Asimismo, es habitual emplear S-boxes de pequeño tamaño y mecanismos de difusión simples basados en permutaciones de bits, reduciendo significativamente el área ocupada y el consumo energético en implementaciones hardware.

Estas características hacen que los cifrados SPN sean especialmente apropiados para plataformas FPGA y sistemas embebidos con recursos limitados, donde la simplicidad estructural y la posibilidad de explotar paralelismo hardware permiten obtener implementaciones eficientes tanto en términos de rendimiento como de utilización de recursos. Un ejemplo representativo de este tipo de algoritmos es PRESENT, un cifrado ligero ampliamente utilizado en el ámbito de la criptografía ligera.

1.3. Plataforma FPGA y entorno de desarrollo

Las implementaciones desarrolladas en este trabajo se ejecutan sobre la plataforma Pynq-Z2, basada en un SoC Zynq-7000 de AMD-Xilinx que integra en un mismo dispositivo un sistema de procesamiento (Processing System, PS) basado en procesadores ARM y una región de lógica programable (Programmable Logic, PL) implementada mediante tecnología FPGA. Esta arquitectura heterogénea permite combinar la flexibilidad de ejecución software con las capacidades de aceleración hardware proporcionadas por la lógica programable.

El sistema de procesamiento ARM se emplea para la ejecución de las implementaciones software de los algoritmos criptográficos, así como para el control general del sistema. Por otra parte, la región FPGA permite implementar aceleradores hardware específicos capaces de ejecutar determinadas operaciones de forma paralela y con menor latencia.

La lógica programable de una FPGA está formada por bloques configurables, principalmente LUTs (Look-Up Tables), que constituyen el elemento básico para la implementación de funciones lógicas. Gracias a estos recursos, es posible configurar circuitos digitales adaptados a distintas aplicaciones y reconfigurar su funcionalidad según las necesidades del sistema.

La comunicación entre el sistema de procesamiento y la lógica programable se realiza mediante interfaces AXI (Advanced eXtensible Interface), utilizadas para transferir datos y señales de control entre ambos dominios. Esta organización permite integrar aceleradores hardware personalizados junto con aplicaciones ejecutadas desde software sobre el procesador ARM.

Para facilitar el desarrollo sobre este tipo de plataformas se emplea PYNQ (Python Productivity for Zynq), un framework de código abierto desarrollado por AMD-Xilinx que permite controlar la lógica programable mediante Python. Gracias a esta aproximación, resulta posible desarrollar aplicaciones híbridas hardware/software de forma más sencilla y accesible, reduciendo la complejidad asociada a los flujos de diseño hardware tradicionales.

La programación y ejecución de las distintas implementaciones se realiza mediante Jupyter Notebook, entorno interactivo integrado en PYNQ. A través de este entorno es posible desarrollar, ejecutar y monitorizar aplicaciones que combinan código software y aceleradores hardware, facilitando además la realización de pruebas experimentales y la obtención de métricas de rendimiento sobre la plataforma.

1.4. Problema abordado y métricas de evaluación

El presente trabajo aborda el análisis y comparación de distintas implementaciones de algoritmos criptográficos de tipo SPN sobre una plataforma heterogénea ARM-FPGA. Para ello, se estudian tanto implementaciones software ejecutadas sobre el procesador ARM de la plataforma como implementaciones hardware desarrolladas sobre la lógica programable de la FPGA. Asimismo, se analiza el impacto de diferentes técnicas de optimización sobre el rendimiento y el consumo de recursos de cada solución.

La evaluación de las distintas implementaciones se realiza empleando diversas métricas de rendimiento y eficiencia. Una de las métricas principales es el throughput o rendimiento, que representa la cantidad de datos cifrados por uni-

dad de tiempo. Otra métrica relevante es la latencia, entendida como el tiempo transcurrido desde que se solicita una operación de cifrado hasta que se obtiene el resultado correspondiente. La importancia relativa de cada métrica depende de la aplicación. El rendimiento tiene importancia cuando se procesa un volumen elevado de datos, mientras que la latencia es importante en aplicaciones que trabajen en tiempo real sobre pocos datos.

Asimismo, el consumo energético constituye uno de los factores más relevantes dentro del ámbito de la criptografía ligera. Muchos de estos dispositivos funcionan mediante baterías de pequeña capacidad o sistemas de alimentación limitados, por lo que minimizar el consumo resulta fundamental para prolongar la autonomía y garantizar la viabilidad del sistema. Este consumo depende en gran medida de factores como la frecuencia de funcionamiento, la complejidad del diseño y la tecnología de implementación empleada.

Además de estas métricas generales, existen recursos críticos específicos para cada tipo de implementación. En las implementaciones hardware adquiere especial relevancia el área ocupada, que en el caso de la FPGA viene determinada por la utilización de recursos como LUTs (Look Up Tables), registros o bloques de memoria. Por otra parte, en las implementaciones software resulta especialmente importante el uso de memoria y la carga computacional asociada a la ejecución del algoritmo sobre el procesador ARM.

El análisis conjunto de estas métricas permite estudiar el compromiso existente entre rendimiento, consumo de recursos y complejidad de implementación, facilitando la identificación de las soluciones más adecuadas para distintos escenarios de aplicación dentro del ámbito de la criptografía ligera y los sistemas embebidos.

1.5. Hipótesis y objetivos

La hipótesis principal de este trabajo plantea que las implementaciones hardware de algoritmos criptográficos de tipo SPN sobre FPGA permiten obtener mejoras significativas en términos de latencia y rendimiento frente a implementaciones software equivalentes ejecutadas sobre procesadores de propósito general, manteniendo un consumo de recursos adecuado para entornos embebidos.

De forma complementaria, se considera que el empleo de técnicas de síntesis de alto nivel (HLS) permite desarrollar aceleradores hardware eficientes y reutilizables, reduciendo la complejidad asociada al diseño hardware tradicional y facilitando su integración dentro de sistemas heterogéneos CPU-FPGA.

Asimismo, se plantea que las distintas técnicas de optimización aplicadas tanto en hardware como en software afectan de manera diferente al equilibrio entre rendimiento, latencia y utilización de recursos, permitiendo obtener soluciones adaptadas a distintos requisitos de diseño.

El objetivo general del trabajo consiste en diseñar, implementar y evaluar

distintas versiones software y hardware de algoritmos de criptografía ligera basados en redes de sustitución y permutación (SPN) sobre la plataforma Pynq-Z2, analizando el impacto de diferentes técnicas de optimización sobre el rendimiento y el consumo de recursos del sistema.

Para alcanzar este objetivo general se plantean los siguientes objetivos específicos:

- Desarrollar una implementación software en Python de un algoritmo SPN simplificado de 32/64 bits ejecutada sobre el procesador ARM de la plataforma Pynq-Z2.
- Diseñar e implementar versiones hardware del algoritmo mediante técnicas de síntesis de alto nivel (HLS) para su ejecución sobre la lógica programable de la plataforma Pynq-Z2.
- Evaluar experimentalmente las distintas soluciones mediante métricas de rendimiento, latencia, consumo energético, utilización de recursos hardware y uso de memoria.
- Aplicar distintas técnicas de optimización hardware y software con el objetivo de analizar su impacto sobre el rendimiento, la latencia y el uso de recursos.
- Integrar las distintas implementaciones hardware dentro de un sistema basado en Pynq utilizando herramientas de diseño como Vitis HLS y Vivado.
- Extender la metodología desarrollada al algoritmo PRESENT-80 como caso representativo de criptografía ligera basada en SPN.
- Ilustrar, mediante una demostración práctica orientada al cifrado de datos reales, la aplicabilidad de las soluciones implementadas en escenarios embebidos.

Capítulo 2

Estado de conocimiento del problema a abordar

2.1. Criptografía ligera y PRESENT

El ámbito de la criptografía ligera ha evolucionado significativamente en los últimos años, impulsado por la necesidad de definir estándares eficientes para dispositivos con recursos limitados. En este contexto, el proceso de estandarización del NIST (National Institute of Standards and Technology) en criptografía ligera ha dado lugar a propuestas recientes como Ascon [4], seleccionado como algoritmo de referencia para cifrado autenticado (Authenticated Encryption with Associated Data, AEAD). Este tipo de soluciones representa el estado actual del arte en el ámbito de la criptografía ligera, priorizando la seguridad y la eficiencia en aplicaciones modernas.

Sin embargo, los algoritmos seleccionados en el proceso del NIST no siempre comparten la misma estructura criptográfica que otras familias clásicas ampliamente estudiadas en el ámbito de la criptografía ligera. En particular, mientras que Ascon se basa en un diseño tipo sponge, otros enfoques históricos han empleado arquitecturas de tipo red de sustitución–permutación (SPN), como es el caso de PRESENT.

PRESENT es uno de los cifrados en bloque más representativos dentro de la criptografía ligera basada en SPN. Propuesto por Bogdanov et al. en 2007 [2], fue diseñado específicamente para entornos con restricciones extremas de hardware y consumo energético, como etiquetas RFID y dispositivos embebidos de muy baja capacidad computacional. Este enfoque lo convirtió en una referencia dentro del diseño de cifrados ligeros eficientes en hardware.

Debido a su estructura SPN y a su simplicidad de implementación, PRESENT se ha convertido en una referencia habitual en trabajos relacionados con criptografía ligera orientada a hardware. En particular, ha sido utilizado en estudios de implementación sobre FPGA, análisis de consumo de recursos, evaluación de

latencia y throughput, así como en propuestas de distintas arquitecturas y optimizaciones hardware [5]. Su diseño compacto, basado en operaciones simples de sustitución y permutación, facilita además la comparación entre distintas estrategias de implementación y evaluación de rendimiento.

En este contexto, PRESENT-80 se emplea en este trabajo como extensión de la metodología desarrollada hacia un caso práctico basado en un cifrado ligero ampliamente utilizado como referencia en la literatura. Se analiza la posible adaptación de las optimizaciones propuestas y aplicabilidad en el contexto de PRESENT, con el objetivo de evaluar su transferibilidad a una arquitectura estandarizada. Este enfoque permite contrastar los resultados obtenidos sobre la plataforma Pynq-Z2 en un escenario representativo y ampliamente estudiado.

2.2. Técnicas de optimización relevantes

Las implementaciones criptográficas, especialmente en el ámbito de la criptografía ligera, presentan un conjunto de técnicas de optimización ampliamente utilizadas tanto en entornos software como hardware. Estas técnicas permiten explorar distintos compromisos entre rendimiento, latencia y utilización de recursos, dependiendo de la plataforma objetivo.

En el ámbito software, destacan técnicas como la implementación mediante tablas de sustitución (lookup tables) o el uso de bit-slicing, que permiten mejorar el rendimiento en arquitecturas de propósito general mediante la explotación del paralelismo a nivel de instrucciones.

En el ámbito hardware, particularmente en implementaciones sobre FPGA, son comunes técnicas como la serialización de operaciones para reducir el consumo de recursos, la construcción de arquitecturas totalmente paralelas para maximizar el rendimiento, y el uso de técnicas de segmentación (pipelining) para incrementar el throughput.

2.3. Síntesis HLS y plataformas heterogéneas

La síntesis de alto nivel (HLS) se ha consolidado en la última década como una alternativa madura a los enfoques tradicionales basados en lenguajes de descripción hardware (HDL). Aunque sus primeras comparaciones sistemáticas con HDL comenzaron a ganar relevancia alrededor de 2015 [6], actualmente constituye una tecnología ampliamente establecida tanto en el ámbito industrial como en investigación, incluyendo aplicaciones recientes en campos como la inteligencia artificial y la computación de altas prestaciones [7, 8].

Este enfoque permite generar implementaciones hardware a partir de lenguajes de alto nivel, facilitando la exploración del espacio de diseño y reduciendo significativamente la complejidad del desarrollo. Como resultado, HLS se ha con-

vertido en una herramienta habitual en el diseño de aceleradores para sistemas embebidos y aplicaciones de alto rendimiento.

En paralelo, la evolución de las plataformas FPGA ha dado lugar a la aparición de arquitecturas heterogéneas tipo SoC, en las que un sistema de procesamiento basado en CPU se integra junto a lógica programable en un mismo dispositivo. Este paradigma comenzó a consolidarse con la introducción de dispositivos como la familia Zynq alrededor de 2010–2012, y ha crecido en paralelo con el desarrollo de herramientas de síntesis de alto nivel, consolidándose como una solución estándar en sistemas embebidos modernos [9].

La combinación de procesadores ARM con lógica FPGA permite distribuir las tareas entre software y hardware, asignando al procesador general el control del sistema y a la FPGA la ejecución de operaciones computacionalmente intensivas. Este modelo resulta especialmente adecuado para aplicaciones criptográficas, donde ciertas tareas pueden beneficiarse de la ejecución concurrente a nivel hardware, mediante técnicas como el paralelismo a nivel de datos, la segmentación de etapas o la replicación de módulos funcionales.

En este contexto, entornos como PYNQ permiten simplificar la interacción con arquitecturas heterogéneas, proporcionando una capa de abstracción que facilita el uso de diseños hardware implementados en la FPGA desde entornos software basados en Python. Esta integración permite controlar y evaluar bloques hardware (IP cores) sin necesidad de gestionar directamente la complejidad del diseño mediante HDL, lo que favorece la exploración rápida de diferentes implementaciones y la validación de distintos diseños en un mismo entorno de ejecución.

2.4. Diferencias con el presente trabajo

En la literatura existente sobre la implementación de cifrados ligeros en FPGA se observa que los trabajos suelen centrarse en una única aproximación de diseño, ya sea hardware o software de forma independiente. En el caso de las implementaciones hardware, el objetivo habitual es la optimización del rendimiento o la reducción del área ocupada en el dispositivo [10], mientras que los estudios software se enfocan principalmente en mejorar el rendimiento sobre procesadores de propósito general [11].

De forma complementaria, también existen trabajos que exploran técnicas de optimización específicas en cada uno de estos ámbitos, tanto mediante síntesis de alto nivel en hardware como mediante optimizaciones a nivel de software [12]. No obstante, estas aproximaciones suelen abordarse de manera aislada, sin establecer un entorno común que permita una comparación directa entre ambas alternativas.

En este contexto, el presente trabajo propone un enfoque unificado que permite el desarrollo y evaluación conjunta de implementaciones software y hardware de algoritmos criptográficos basados en redes de sustitución–permutación (SPN),

utilizando una plataforma heterogénea basada en PYNQ-Z2. Este enfoque permite realizar comparaciones bajo condiciones controladas, reduciendo las diferencias derivadas de entornos de ejecución diferentes.

Además, se adopta un flujo de diseño basado en HLS, que facilita la generación de distintas versiones hardware mediante estrategias de optimización. Estas implementaciones se integran como bloques reutilizables dentro de un entorno común, permitiendo su evaluación en términos de rendimiento, latencia y utilización de recursos.

Los diseños hardware generados pueden ser encapsulados como bloques IP integrables en flujos de diseño basados en herramientas como Vivado, lo que facilita su reutilización y su adaptación a distintos escenarios de implementación. Finalmente, el sistema se valida mediante una demostración práctica orientada al cifrado de datos reales, lo que refuerza su aplicabilidad en entornos embebidos.

En conjunto, este trabajo unifica el diseño, la optimización y la evaluación de implementaciones criptográficas hardware y software dentro de una misma plataforma heterogénea.

Capítulo 3

Metodología

3.1. Materiales

3.1.1. Plataforma hardware

El desarrollo experimental del trabajo se realizó sobre la plataforma Pynq-Z2, basada en el dispositivo Zynq-7000 XC7Z020 de AMD-Xilinx. La plataforma integra un sistema de procesamiento ARM Cortex-A9 de doble núcleo junto con lógica programable FPGA, permitiendo la ejecución conjunta de aplicaciones software y aceleradores hardware.

Las principales características de la placa, incluyendo memoria e interfaces de comunicación, se resumen en la Tabla 3.1.

Aspecto	**Especificación**
SoC	Xilinx Zynq XC7Z020
Procesador	Dual-core ARM Cortex-A9 @ 667 MHz
Lógica programable	85K logic cells / 106K FFs / 220 DSPs
Memoria	512 MB DDR3 (16-bit @ 1050 Mbps)
Memoria Flash	128 Mbit Quad-SPI NOR (16 MB)
Almacenamiento	Ranura MicroSD
Ethernet	Gigabit Ethernet (10/100/1000 Mbps)
Interfaces USB	USB OTG 2.0, UART y JTAG
Expansión	2× Pmod, Arduino Shield, HAT Raspberry Pi
E/S	4 LEDs, 4 pulsadores, 2 switches y 6 entradas analógicas
Alimentación	USB micro-B o fuente externa de 5V
Entorno de desarrollo	PYNQ, Vivado y Vitis

Tabla 3.1: Especificaciones de la placa PYNQ-Z2 [14].

3.1.2. Herramientas de desarrollo

Las implementaciones software se desarrollaron en Python utilizando el entorno Jupyter Notebook proporcionado por PYNQ.

Para el desarrollo hardware se empleó un flujo basado en síntesis de alto nivel mediante Vitis HLS, utilizando descripciones en C/C++. La integración de los aceleradores hardware dentro del sistema completo se realizó mediante Vivado.

Finalmente, las distintas configuraciones hardware fueron desplegadas sobre la FPGA mediante los bitstreams generados durante el proceso de síntesis e implementación.

3.1.3. Entorno experimental

La ejecución de las pruebas y la obtención de métricas experimentales se realizaron desde el entorno Jupyter Notebook ejecutado sobre el procesador ARM de la plataforma. Este entorno permitió controlar tanto las implementaciones software como los aceleradores hardware integrados en la FPGA desde una misma interfaz de ejecución.

3.2. Implementaciones Software

3.2.1. Implementación básica

Como punto de partida se desarrolló una implementación básica de un cifrado de tipo SPN de bloque reducido (32 bits) y clave de 64 bits. Esta primera versión siguió una estructura iterativa basada en rondas, incorporando las operaciones fundamentales características de los cifrados SPN: sustitución mediante S-boxes, permutación de bits y adición de clave mediante operaciones XOR.

La etapa de sustitución se implementó aplicando una misma tabla de sustitución sobre cada bloque de 4 bits del estado interno. La S-box empleada toma como referencia la utilizada por el algoritmo PRESENT, misma que será empleada en el resto de implementaciones y se recoge en la Tabla 3.2.

input	0	1	2	3	4	5	6	7
output	C	5	6	B	9	0	A	D

input	8	9	A	B	C	D	E	F
output	3	E	F	8	4	7	1	2

Tabla 3.2: Mapeo de la S-box de 4 bits de PRESENT

Por otra parte, la operación de permutación se implementó mediante una estrategia de rotación y recolocación de bits (bit rotation). Este procedimiento

consiste en procesar cada bit del estado de forma individual: inicialmente se aísla utilizando operaciones de enmascaramiento, posteriormente se desplaza hasta la posición correspondiente dentro del bloque de salida y, finalmente, se combina con el resultado acumulado mediante operaciones lógicas. La posición final de cada bit se define mediante la función de mapeo recogida en la Ecuación 3.1.

$$\text{pos}(i) = \left\lfloor \frac{i}{4} \right\rfloor + N_{\text{nibbles}} \cdot (i \bmod 4), \quad i \in \{0, 1, \ldots, n-1\}$$

Ecuación 3.1: Función de mapeo para la posición de los bits

La implementación inicial se diseñó priorizando la simplicidad y claridad estructural, con el objetivo de disponer de una referencia funcional sobre la que aplicar posteriormente distintas técnicas de optimización software. Esta versión permitió además validar el correcto funcionamiento del algoritmo y establecer una base de comparación homogénea para el resto de implementaciones desarrolladas.

A partir de esta implementación básica se desarrollaron distintas versiones optimizadas orientadas a mejorar el rendimiento de ejecución sobre CPU.

3.2.2. Implementaciones basadas en tablas de búsqueda

Una de las técnicas empleadas fue la implementación mediante tablas precalculadas. Este enfoque consiste en reemplazar determinadas operaciones del algoritmo por accesos directos a valores almacenados en memoria. De este modo, en lugar de calcular dinámicamente el resultado de una función durante el proceso de cifrado, dicho resultado se obtiene directamente a partir de una tabla indexada mediante la entrada correspondiente, reduciendo así el número de operaciones ejecutadas por el procesador.

En los cifrados SPN esta técnica resulta especialmente adecuada debido a la naturaleza repetitiva de las operaciones de sustitución y permutación. Además, permite explorar distintos compromisos entre consumo de memoria y rendimiento, modificando el tamaño y complejidad de las tablas empleadas. A partir de esta idea general se desarrollaron distintas variantes optimizadas de forma incremental.

La primera optimización aplicada consistió en implementar la operación de permutación mediante tablas de consulta. Teóricamente sería posible utilizar una única tabla capaz de almacenar directamente el resultado completo de la permutación para cualquier estado de 32 bits. Sin embargo, una aproximación de este tipo requeriría una cantidad de memoria excesiva, del orden de 2^{32} entradas. Por este motivo, la implementación realizada divide el estado en bloques de 4 bits (nibbles), utilizando tablas independientes que almacenan únicamente la contri-

bución permutada correspondiente a cada uno de estos bloques, como se muestra en la Tabla 3.3.

Input	*Output(hex)*	*Input*	*Output(hex)*
0000	00000000	1000	01000000
0001	00000001	1001	01000001
0010	00000100	1010	01000100
0011	00000101	1011	01000101
0100	00010000	1100	01010000
0101	00010001	1101	01010001
0110	00010100	1110	01010100
0111	00010101	1111	01010101

Tabla 3.3: Tabla de permutación para los 4 bits menos significativos

La siguiente variante desarrollada combina las etapas de sustitución y permutación en una única operación mediante tablas SP (Substitution-Permutation tables). En este caso, cada tabla contiene directamente el resultado de aplicar de forma sucesiva las operaciones de sustitución y permutación sobre cada nibble de entrada, reduciendo así el número total de operaciones necesarias durante el cifrado.

A partir de esta implementación se desarrolló una versión basada en el uso de una única tabla SP. En lugar de almacenar múltiples tablas independientes, únicamente se mantiene una de ellas en memoria, calculándose el resto de resultados mediante operaciones de desplazamiento sobre los valores obtenidos. Esta aproximación permite reducir significativamente el consumo de memoria a costa de introducir un pequeño número adicional de operaciones lógicas.

Finalmente, se implementó una variante basada en la combinación de tablas SP de mayor tamaño. En lugar de procesar bloques de 4 bits, estas versiones permiten realizar búsquedas directas sobre grupos mayores de bits, reduciendo el número total de accesos a memoria necesarios durante el cifrado. No obstante, el incremento del tamaño de las tablas implica también un mayor consumo de memoria y puede llegar a afectar negativamente al rendimiento si el coste de acceso a memoria supera la reducción de operaciones conseguida.

3.2.3. Implementación basada en *bit-slicing*

Se desarrolló una implementación basada en *bit-slicing*, técnica utilizada en criptografía software para realizar en paralelo las operaciones lógicas de un cifrado SPN sobre varios bloques simultáneamente, aprovechando el paralelismo a nivel de bits de los registros del procesador.

En la implementación desarrollada se trabajó de forma simultánea sobre conjuntos de 32 bloques de texto plano. Para ello, fue necesario reorganizar previa-

mente la información en formato bit-slice, de forma matricial, como se describe en la Ecuación 3.2. Inicialmente, cada bloque se representa de forma convencional, almacenándose un bloque completo por fila. Posteriormente, se realiza una transposición de los datos de forma que cada fila pasa a contener el mismo bit de los distintos bloques procesados. De este modo, cada operación lógica aplicada sobre una fila afecta simultáneamente al bit correspondiente de los 32 bloques.

Una vez completado el proceso de cifrado, se realiza la operación inversa para reconstruir los bloques cifrados en formato convencional. Por tanto, la implementación bit-slice incorpora dos etapas adicionales de reorganización de datos: una previa al cifrado y otra posterior a este.

$$P = \begin{bmatrix} P_{31} \\ P_{30} \\ \vdots \\ P_0 \end{bmatrix} = \begin{bmatrix} p_{31,31} & p_{31,30} & \cdots & p_{31,0} \\ p_{30,31} & p_{30,30} & \cdots & p_{30,0} \\ \vdots & \vdots & \ddots & \vdots \\ p_{0,31} & p_{0,30} & \cdots & p_{0,0} \end{bmatrix} \xrightarrow{\text{Permutación}} D = \begin{bmatrix} D_{31} \\ D_{30} \\ \vdots \\ D_0 \end{bmatrix} = \begin{bmatrix} p_{31,31} & p_{30,31} & \cdots & p_{0,31} \\ p_{31,30} & p_{30,30} & \cdots & p_{0,30} \\ \vdots & \vdots & \ddots & \vdots \\ p_{31,0} & p_{30,0} & \cdots & p_{0,0} \end{bmatrix}$$

Ecuación 3.2: Representación matricial de la permutación bit-slice sobre el estado del cifrado.

El uso de esta representación modifica además la forma en que se implementan las operaciones fundamentales del cifrado SPN. La etapa de sustitución deja de realizarse mediante tablas de consulta, pasando a expresarse como una secuencia de operaciones lógicas booleanas. En este contexto, uno de los principales objetivos consiste en minimizar el número de operaciones necesarias para implementar la S-box. En el caso de la S-box empleada por PRESENT, existen expresiones booleanas optimizadas ya conocidas.

La operación de permutación también se adapta al nuevo formato de representación, realizándose sobre conjuntos completos de bits en lugar de operar individualmente sobre cada bit del estado. De forma similar, la incorporación de clave mediante XOR se lleva a cabo directamente entre las filas correspondientes de la representación bit-slice.

Aunque esta técnica introduce una mayor complejidad en la reorganización de datos y en la implementación interna del algoritmo, permite aprovechar de forma eficiente el paralelismo con operaciones lógicas, mejorando significativamente el rendimiento en determinadas arquitecturas software.

De forma transversal a todas las implementaciones software desarrolladas, se consideraron dos estrategias alternativas para la generación de subclaves (round keys). En la primera aproximación, las subclaves se calculan dinámicamente durante la ejecución del algoritmo, generándose la clave correspondiente a cada ronda justo antes de su utilización.

En la segunda aproximación, el conjunto completo de subclaves se calcula previamente a la ejecución del cifrado y se almacena en memoria en una estructura de tipo array, permitiendo su reutilización directa en cada ronda.

Ambas estrategias permiten explorar el compromiso existente entre consumo de memoria y rendimiento computacional, siendo su idoneidad dependiente del tipo de implementación y de las restricciones del entorno de ejecución.

3.3. Implementaciones Hardware

3.3.1. Flujo de implementación hardware

Todas las variantes hardware desarrolladas en este trabajo siguen un flujo de diseño común basado en tres herramientas principales: Vitis HLS, Vivado y el framework PYNQ. Este flujo permite partir de una descripción funcional del algoritmo y llegar hasta su ejecución sobre la lógica programable de la plataforma Pynq-Z2.

La primera etapa del flujo de diseño se realiza mediante Vitis HLS, herramienta que permite describir el comportamiento del sistema en lenguaje C/C++ y sintetizarlo posteriormente en hardware. En esta fase se emplean directivas de síntesis (pragmas) que permiten guiar la generación del circuito hardware, definiendo aspectos como el nivel de paralelismo, la estructura de los bucles o la forma de implementación de determinadas operaciones.

```
uint32 cifrador_basico(uint32 plaintext, uint64 key) {
    #pragma HLS INTERFACE s_axilite port=plaintext
    #pragma HLS INTERFACE s_axilite port=key
    #pragma HLS INTERFACE s_axilite port=return
}
```

Código 3.1: Ejemplo de directivas HLS para definir interfaces I/O tipo AXI-Lite

Un aspecto fundamental en esta etapa es la definición de las interfaces de entrada y salida del módulo hardware, que determinan cómo se comunica el bloque sintetizado con el exterior. En este trabajo se utilizan principalmente interfaces de tipo AXI-Lite, adecuadas para la transferencia de datos de control y para la interacción con registros internos del acelerador.

Como resultado de este proceso se obtiene un núcleo hardware sintetizable (IP core), junto con los identificadores de los registros de entrada y salida necesarios para su posterior integración y control desde el entorno software.

En la siguiente etapa, el bloque IP generado mediante Vitis HLS se integra en un diseño hardware completo utilizando la herramienta Vivado. Este entorno permite construir la arquitectura del sistema sobre la FPGA, conectando el bloque IP con el sistema de procesamiento ARM y con el resto de componentes de la placa mediante buses AXI.

Una vez completada la integración del sistema, se realiza el proceso de síntesis e implementación, obteniéndose como resultado final un bitstream que configura la lógica programable de la FPGA.

Finalmente, el bitstream generado junto con los archivos auxiliares del diseño se importan al entorno PYNQ, donde se cargan como un overlay sobre la plataforma Pynq-Z2. Este mecanismo permite exponer el hardware sintetizado como un conjunto de recursos accesibles desde Python.

Para facilitar la interacción con el acelerador hardware, se implementan controladores (*drivers*) en el entorno Jupyter Notebook. Estos drivers permiten abstraer el acceso a los registros del dispositivo mediante llamadas a funciones de alto nivel.

Cada controlador implementa tres operaciones principales: escritura de datos de entrada en los registros del acelerador, ejecución del proceso de cifrado mediante una señal de inicio, y lectura del resultado final. La sincronización entre el procesador y la lógica programable se realiza mediante un mecanismo de polling, en el cual el procesador consulta periódicamente el estado del acelerador hasta la finalización del proceso. Esta estrategia se adopta por su simplicidad y porque la ejecución del cifrado en la lógica programable no interfiere en la carga computacional del procesador, permitiendo además una medición directa del rendimiento del hardware.

3.3.2. Implementacion básica

La primera implementación hardware del cifrado SPN se desarrolló como una versión equivalente a la implementación software básica, con el objetivo de establecer una referencia inicial para la evaluación comparativa de rendimiento entre ambas plataformas.

En esta versión inicial, las operaciones de sustitución, permutación y adición de clave se ejecutan de forma secuencial dentro de cada ronda del cifrado, sin aplicar técnicas explícitas de paralelización o segmentación.

Esta implementación proporciona una base funcional del cifrado sobre FPGA, directamente comparable con la versión software, y sirve como punto de partida para el análisis del impacto de las optimizaciones hardware introducidas en fases posteriores del diseño.

3.3.3. Implementaciones basadas en *loop unrolling*

La primera técnica de optimización aplicada sobre la implementación hardware consiste en la utilización de *loop unrolling*, implementada mediante la directiva `#pragma HLS unroll` proporcionada por Vitis HLS. Esta técnica permite reducir o eliminar la estructura iterativa de un bucle mediante la replicación del cuerpo del mismo, con el objetivo de aumentar el grado de paralelismo.

En el contexto del cifrado SPN, esta optimización se aplica sobre el bucle principal que controla la ejecución de las distintas rondas del algoritmo, permitiendo que múltiples rondas del cifrado puedan ser evaluadas de manera concurrente en función del factor de desenrollado seleccionado.

De este modo, en lugar de ejecutar las rondas de forma estrictamente secuencial, el diseño puede procesar varias de ellas en paralelo dentro de un mismo ciclo de reloj, incrementando el número de operaciones realizadas por unidad de tiempo y mejorando potencialmente el rendimiento global del sistema.

Se han evaluado diferentes factores de desenrollado con el objetivo de analizar su impacto sobre el rendimiento y el consumo de recursos hardware, observándose cómo el grado de paralelización introducido influye directamente en la relación entre latencia, ocupación de la FPGA y throughput del sistema.

3.3.4. Implementaciones basadas en paralelización

La segunda técnica de optimización aplicada consiste en la paralelización a nivel de arquitectura, mediante la replicación del bloque hardware del cifrador con el objetivo de ejecutar múltiples instancias del algoritmo de forma concurrente. Esta estrategia permite incrementar el throughput del sistema, ya que se procesan varios bloques de datos en paralelo dentro de la lógica programable de la FPGA.

A diferencia de otras optimizaciones orientadas a mejorar la eficiencia interna de un único cifrador, esta técnica se basa en la duplicación del sistema completo, generando múltiples canales de ejecución independientes. El grado de paralelización puede ajustarse en función de los recursos disponibles en la FPGA, estando limitado principalmente por el área de lógica ocupada y los canales de comunicación.

En este trabajo se han implementado distintas configuraciones con diferentes niveles de replicación del cifrador, con el objetivo de analizar la relación entre el aumento de rendimiento y el coste en términos de utilización de recursos hardware.

La replicación del cifrador introduce múltiples instancias concurrentes del algoritmo, incluyendo la generación de subclaves (*key schedule*). Se han explorado dos estrategias para extraer los generadores de subclaves de los cifradores y que exista como un único bloque independiente compartido por cada uno de los cifradores, como se representa en la Figura 3.1.

En una primera aproximación, se opta por el cálculo previo de todas las subclaves antes del inicio del proceso de cifrado. Estas subclaves se almacenan en registros o estructuras de memoria accesibles por cada instancia del cifrador, permitiendo su reutilización durante la ejecución paralela sin necesidad de recomputación.

Una segunda aproximación introduce una mayor integración entre el *key schedule* y los cifradores replicados, utilizando una comunicación de tipo streaming. En este caso, el generador de subclaves actúa como productor de datos, enviando de forma secuencial cada subclave a través de colas FIFO independientes asociadas a cada cifrador. Cada instancia consumidora recibe las subclaves conforme las necesita, permitiendo la ejecución concurrente del proceso de generación de claves y cifrado.

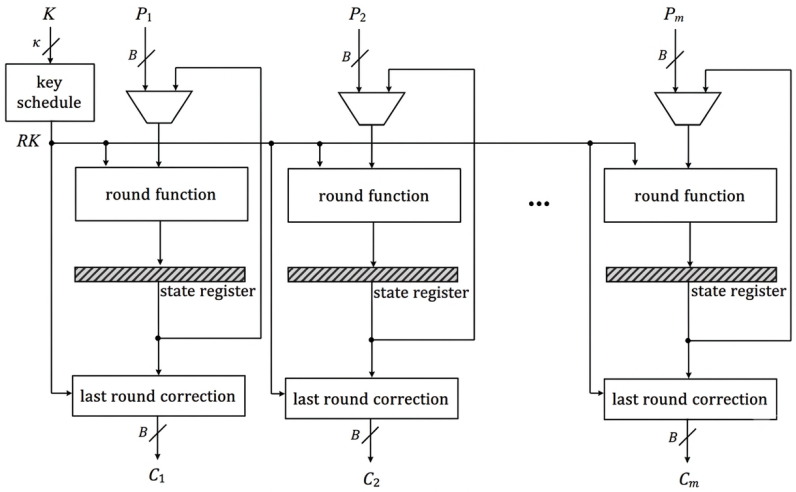


Figura 3.1: Ejemplo de diseño paralelo [1]

Además de las distintas variantes paralelas estudiadas, se empleará esta arquitectura para implementar un modo de operación de cifrado seguro: el modo CTR (Counter Mode). Aunque existen múltiples modos de operación capaces de evitar los problemas de seguridad asociados al cifrado independiente de bloques, CTR resulta especialmente interesante por ser ampliamente utilizado en la actualidad y por su alta compatibilidad con arquitecturas paralelas.

En un cifrado SPN básico operando en modo ECB (Electronic Codebook), donde los bloques son procesados de manera independiente, bloques de texto plano idénticos producen bloques cifrados idénticos, revelando patrones presentes en la información procesada. El modo CTR evita este problema combinando cada bloque con un contador diferente antes del proceso de cifrado, garantizando que incluso bloques iguales generen resultados distintos. Además, dado que cada bloque puede procesarse de forma independiente, CTR permite mantener las ventajas de rendimiento proporcionadas por la paralelización, sin introducir dependencias entre bloques que dificulten la ejecución simultánea de múltiples cifradores. De esta forma, esta implementación permite ilustrar no solo cómo ciertas optimizaciones afectan al rendimiento, sino también cómo determinadas decisiones arquitectónicas pueden condicionar la elección de modos de operación criptográficos seguros.

3.3.5. Implementaciones basadas en segmentación (*pipelining*)

La tercera técnica de optimización aplicada consiste en la utilización de segmentación (*pipelining*), una estrategia de diseño hardware que permite descomponer el cifrado en múltiples etapas secuenciales conectadas mediante registros

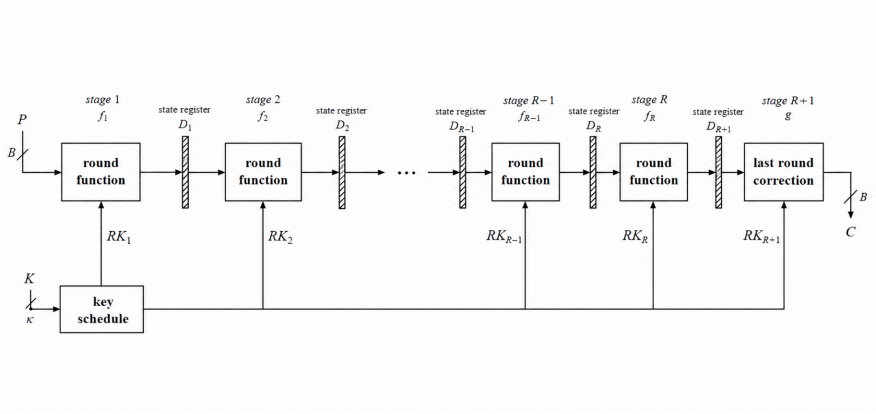


Figura 3.2: Ejemplo de diseño pipeline [1]

intermedios, tal y como se muestra en la Figura 3.2. Esta estructura facilita la ejecución solapada de diferentes fases del algoritmo, incrementando el rendimiento global del sistema.

En la implementación desarrollada, el cifrador se divide en un conjunto de etapas asociadas a las distintas rondas del algoritmo SPN. En la configuración inicial, se implementa un pipeline profundo compuesto por 32 etapas, correspondientes a las rondas del cifrado, donde la etapa final implementa la corrección final del esquema SPN.

Asimismo, se han explorado variantes en las que varias rondas del cifrado se agrupan dentro de una misma etapa del pipeline, con el objetivo de estudiar el compromiso entre rendimiento y utilización de recursos hardware.

Desde el punto de vista del comportamiento temporal, cada etapa del pipeline requiere un ciclo de reloj para completar su operación. En consecuencia, la latencia total de procesamiento de un bloque corresponde aproximadamente al número de etapas del diseño, separadas por registros intermedios. Sin embargo, una vez que el pipeline se encuentra completamente lleno, el sistema es capaz de producir un nuevo bloque cifrado en cada ciclo de reloj, lo que incrementa significativamente el throughput del sistema.

De este modo, aunque la latencia inicial del primer bloque aumenta, el rendimiento global del sistema mejora sustancialmente al mantener un flujo continuo de bloques en procesamiento.

3.3.6. Implementación basada en serialización

La última estrategia de optimización explorada se orienta a la reducción del área ocupada por el diseño hardware. Para ello, se implementa una arquitectura serializada basada en la reutilización de una única instancia de la S-box durante todo el proceso de cifrado (véase Figura 3.3).

Con este objetivo, se emplea la directiva de síntesis:

```
#pragma HLS ALLOCATION function instances=sbox limit=1
```

Esta directiva garantiza que únicamente se sintetice una instancia hardware de la función correspondiente a la S-box, evitando que la herramienta HLS genere múltiples copias automáticas durante el proceso de síntesis.

A diferencia de las arquitecturas paralelas anteriores, donde múltiples sustituciones podían ejecutarse simultáneamente, en este diseño los distintos *nibbles* del estado interno se procesan secuencialmente reutilizando la misma unidad funcional. Para ello, se implementa un registro de estado seleccionable encargado de almacenar temporalmente el bloque en proceso de cifrado y permitir el acceso individual a cada uno de sus *nibbles*.

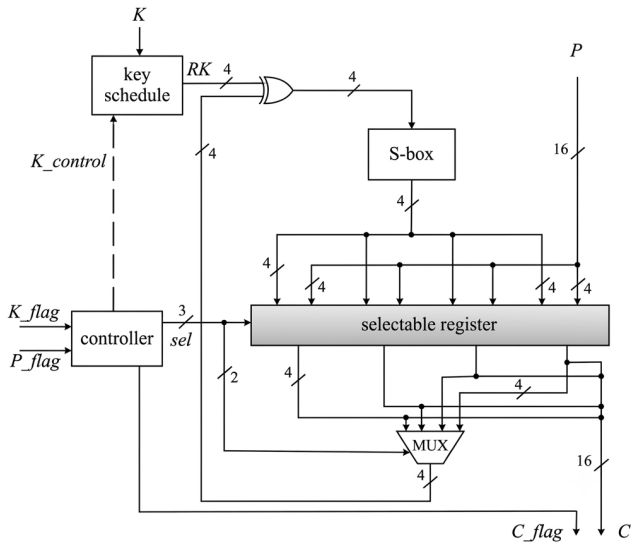


Figura 3.3: Ejemplo de diseño serializado [1]

Durante la ejecución de cada ronda, el sistema selecciona secuencialmente cada *nibble* del estado, lo combina mediante una operación XOR con la porción correspondiente de la subclave y aplica posteriormente la transformación de sustitución mediante la S-box compartida. El resultado obtenido se vuelve a almacenar en el registro de estado antes de continuar con el siguiente *nibble*.

Una vez completado el procesamiento de todos los *nibbles* del bloque, se realiza la operación de permutación sobre el registro completo, finalizando así la ronda correspondiente del cifrado.

3.4. PRESENT-80

Una vez desarrolladas las distintas implementaciones del cifrado SPN propuesto, la metodología se extendió al algoritmo PRESENT-80 con el objetivo de

validar su aplicabilidad sobre un caso práctico utilizado dentro del ámbito de la criptografía ligera.

La adaptación de las implementaciones desarrolladas al algoritmo PRESENT-80 resultó relativamente directa, debido a la similitud estructural existente entre ambos cifrados. En esencia, el proceso consistió en ajustar diversos parámetros del diseño para que coincidiesen con las especificaciones descritas en la literatura para PRESENT.

En primer lugar, se modificó el tamaño del bloque de datos, pasando de 32 a 64 bits. Como consecuencia, aumentó también el número de S-boxes necesarias en cada ronda, manteniéndose en cualquier caso el uso de cajas de sustitución de 4 bits características del algoritmo PRESENT.

El tamaño de la clave principal se amplió de 64 a 80 bits, adaptando consecuentemente el mecanismo de generación de subclaves. En este sentido, fue necesario modificar el algoritmo de expansión de clave para ajustarlo al esquema definido en la especificación original de PRESENT-80.

Por otra parte, la operación de sustitución empleada en las implementaciones SPN ya utilizaba la misma S-box definida por PRESENT, por lo que no fue necesario realizar modificaciones sobre esta etapa. De forma similar, la operación de permutación seguía el mismo patrón estructural que el algoritmo PRESENT, siendo suficiente adaptar el tamaño de bloque manejado por dicha transformación.

Se implementaron las versiones básicas software y hardware del algoritmo PRESENT-80. Estas implementaciones sirven como referencia para analizar en qué medida las métricas obtenidas sobre el cifrado SPN simplificado se aproximan al comportamiento observado en un algoritmo de criptografía ligera real estudiado en la literatura.

Capítulo 4

Pruebas

Las pruebas realizadas tienen como objetivo validar el correcto funcionamiento de los cifradores implementados y, especialmente, garantizar la correspondencia funcional entre las distintas variantes desarrolladas, tanto software como hardware. En particular, se busca comprobar que, dado un mismo bloque de texto plano y una misma clave de cifrado, todas las implementaciones generan exactamente el mismo bloque cifrado, independientemente de las optimizaciones o diferencias estructurales introducidas en cada diseño. De este modo, se verifica que todas las variantes implementadas responden correctamente al mismo algoritmo SPN definido inicialmente.

Para validar el funcionamiento básico del sistema, tras desarrollar la implementación software de referencia se implementó también el descifrador correspondiente. Esto permitió comprobar que, dado un bloque cifrado y empleando la misma clave utilizada durante el proceso de cifrado, es posible recuperar correctamente el bloque de texto original. Esta validación inicial permitió confirmar la correcta implementación de las operaciones fundamentales del esquema SPN.

A partir de esta implementación de referencia se generó un conjunto de casos de prueba o*test vectors*, compuestos por tripletes formados por un bloque de texto plano, una clave de cifrado y el bloque cifrado esperado como resultado (véase Tabla 4.1). Estos vectores de prueba fueron posteriormente utilizados para validar todas las variantes implementadas, tanto software como hardware.

Texto Plano	*Llave*	*Cifrado Esperado*
01234567	01234567 89ABCDEF	F9097518
00000000	00000000 00000000	2D15D21E
FFFFFFFF	FFFFFFFF FFFFFFFF	459ECD3C

Tabla 4.1: Vectores de prueba para SPN (en formato hexadecimal).

Durante el proceso de validación se verificó que todas las implementaciones analizadas producían exactamente los mismos resultados para los distintos casos

de prueba considerados.

En el caso de las implementaciones correspondientes al algoritmo PRESENT-80, la validación funcional se realizó empleando vectores de prueba públicos extraídos de la especificación y documentación disponible en la literatura [2]. Al igual que en los casos de prueba generados para el cifrado SPN desarrollado, estos vectores están compuestos por un bloque de texto plano, una clave de cifrado y el bloque cifrado esperado como resultado (véase Tabla 4.2). Mediante estos casos de prueba se comprobó que las implementaciones desarrolladas producían los resultados correctos definidos para el algoritmo PRESENT-80.

Texto Plano	*Llave*	*Cifrado Esperado*
00000000 00000000	00000000 00000000 0000	5579C138 7B228445
00000000 00000000	FFFFFFFF FFFFFFFF FFFF	E72C46C0 F5945049
FFFFFFFF FFFFFFFF	00000000 00000000 0000	A112FFC7 2F68417B
FFFFFFFF FFFFFFFF	FFFFFFFF FFFFFFFF FFFF	3333DCD3 213210D2

Tabla 4.2: Vectores de prueba para PRESENT-80 (en formato hexadecimal).

Adicionalmente, se aplicaron distintas medidas orientadas a garantizar la validez y estabilidad de las métricas experimentales obtenidas. Tal y como se describió en la metodología, antes de realizar las mediciones se ejecutaron fases de *warm-up* destinadas a reducir el impacto de las primeras ejecuciones y estabilizar el comportamiento del sistema. Asimismo, todas las métricas se calcularon a partir de múltiples ejecuciones consecutivas, utilizando posteriormente el valor medio de los resultados obtenidos con el fin de reducir la influencia del ruido introducido por el sistema operativo y otros procesos en segundo plano.

En el caso de las implementaciones hardware, se tomó además la precaución de medir exclusivamente el tiempo correspondiente a la ejecución interna del acelerador criptográfico, evitando incluir operaciones auxiliares realizadas por el procesador ARM, como la escritura inicial de registros o la lectura de resultados. De este modo, se evita introducir incrementos artificiales en las medidas de latencia y distorsiones en las estimaciones de throughput obtenidas para los distintos diseños hardware.

Capítulo 5

Discusión de los resultados

5.1. Métricas

Con el objetivo de evaluar y comparar las distintas implementaciones desarrolladas, se definieron un conjunto de métricas orientadas tanto al análisis de rendimiento como al consumo de recursos. Concretamente, las métricas consideradas en este trabajo son latencia, throughput o rendimiento, consumo de memoria, consumo de potencia y utilización de área hardware.

Las medidas de latencia, rendimiento y consumo de memoria se obtuvieron desde el entorno Jupyter Notebook de PYNQ mediante funciones específicas desarrolladas en Python para cada métrica. Estas funciones hacen uso de las utilidades proporcionadas por el lenguaje para la medición temporal y el análisis del uso de memoria durante la ejecución de los algoritmos.

Todas las funciones de medida comparten una metodología común orientada a reducir la variabilidad experimental y mejorar la estabilidad de los resultados obtenidos.

En primer lugar, antes de comenzar las mediciones se ejecuta una fase de *warm-up* consistente en varias ejecuciones preliminares del algoritmo. Esta etapa permite reducir el impacto asociado a las primeras ejecuciones, normalmente más lentas debido a fenómenos relacionados con la inicialización del entorno de ejecución. Entre estos efectos destacan el calentamiento de las cachés L1 y L2 mediante la carga de instrucciones y datos, así como la estabilización de accesos a memoria tras la carga de entradas en la TLB (Translation Lookaside Buffer).

En segundo lugar, las métricas se calculan a partir de múltiples ejecuciones consecutivas del cifrado, obteniendo posteriormente el valor medio de los resultados registrados. Este procedimiento permite reducir el ruido experimental introducido por el sistema operativo Linux ejecutado sobre la plataforma, incluyendo efectos asociados al planificador de procesos, interrupciones del sistema, controladores y otros servicios ejecutados en segundo plano.

El área ocupada y el consumo de potencia se obtienen a partir de las esti-

maciones proporcionadas por los informes generados por la herramienta Vivado tras el proceso de síntesis e implementación del diseño hardware, y por tanto corresponden exclusivamente a las implementaciones hardware.

5.1.1. Latencia

La latencia mide el tiempo necesario para que un bloque de datos atraviese completamente todas las etapas del algoritmo de cifrado. En las implementaciones software, este valor coincide directamente con el tiempo de ejecución de una llamada individual a la función de cifrado.

En el caso de las implementaciones hardware, las medidas se realizan exclusivamente sobre la etapa de ejecución del acelerador. Para ello, los controladores desarrollados separan explícitamente las operaciones de escritura de datos de entrada, activación del cifrado y lectura del resultado. De este modo, el tiempo asociado a la transferencia de datos entre el procesador ARM y la lógica programable no influye sobre la medición de la latencia propia del diseño hardware.

5.1.2. Throughput

El throughput o rendimiento mide la cantidad de bloques procesados por unidad de tiempo.

Al igual que en la medida de latencia, en las implementaciones hardware únicamente se considera el tiempo correspondiente a la ejecución, excluyendo las operaciones auxiliares de transferencia de datos realizadas por el procesador.

5.1.3. Consumo de memoria

El análisis del consumo de memoria se realiza exclusivamente sobre las implementaciones software. En este contexto, se consideran dos componentes principales. Por un lado, la memoria pico utilizada durante la ejecución del cifrado, la cual proporciona una aproximación de los recursos dinámicos necesarios para la ejecución del algoritmo. Por otro lado, se tiene en cuenta la memoria estática asociada a estructuras auxiliares, como tablas de sustitución o tablas de permutación en aquellas variantes que las emplean, la cual representa el coste fijo de almacenamiento del algoritmo.

Para mejorar la precisión de las medidas dinámicas, se libera la memoria utilizada entre ejecuciones consecutivas antes de realizar una nueva medición, evitando así la acumulación de estructuras temporales generadas durante pruebas anteriores. Este procedimiento permite obtener una estimación más estable de la memoria pico, reduciendo el impacto de residuos de ejecuciones previas.

5.1.4. Área ocupada

El área ocupada se refiere a la porción de la lógica programable de la FPGA utilizada por cada diseño hardware. En lugar de considerarse una única magnitud global, Vivado descompone esta métrica en distintos tipos de recursos hardware, permitiendo analizar con mayor precisión el impacto de cada implementación.

Entre los recursos más relevantes se encuentran las LUTs (Look-Up Tables), que constituyen el elemento básico de la lógica programable. Estas memorias configurables implementan funciones lógicas combinacionales y representan una parte fundamental del diseño digital en FPGA.

Otro recurso importante son los flip-flops (FF), que constituyen la unidad básica de almacenamiento de un bit. Se utilizan principalmente en registros y elementos de control secuencial, siendo esenciales para la implementación de estados internos del sistema.

Asimismo, se consideran las LUTRAM, que corresponden a LUTs configuradas como pequeñas memorias distribuidas. Estas estructuras suelen emplearse como buffers de baja capacidad o tablas de acceso rápido dentro del diseño.

Finalmente, destacan las BRAM (Block RAM), que constituyen bloques de memoria dedicados dentro de la FPGA. A diferencia de las memorias distribuidas, las BRAM permiten almacenar mayores cantidades de información y se utilizan en estructuras de datos más pesadas o buffers de mayor tamaño.

5.1.5. Consumo de potencia

El consumo de potencia se obtiene también a partir de las estimaciones proporcionadas por Vivado, que desglosa el consumo energético total del sistema en distintos componentes. En particular, se distingue entre potencia estática y potencia dinámica.

La potencia estática corresponde al consumo base del dispositivo cuando no existe actividad de conmutación significativa, mientras que la potencia dinámica está asociada a la actividad del circuito durante su funcionamiento.

Dentro de la potencia dinámica, Vivado permite diferenciar el consumo asociado al procesador, a la lógica programable, a las señales de control y a las señales de reloj, proporcionando una visión detallada del comportamiento energético del sistema implementado.

Aunque sería deseable disponer de una medición equivalente para las implementaciones software ejecutadas sobre el procesador ARM, en este trabajo no ha sido posible realizar una estimación precisa de dicho consumo. Esto se debe tanto a la ausencia de herramientas de perfilado energético adecuadas para el entorno utilizado como a la falta de instrumentación física en la placa capaz de registrar el consumo real del sistema durante la ejecución.

5.2. Resultados de las métricas software

5.2.1. Latencia

Los valores de latencia obtenidos para las distintas implementaciones software, expresados en milisegundos, pueden consultarse en la Tabla 5.1. En estas medidas, al igual que en el resto de experimentos realizados en el ámbito software, se distinguen dos variantes de cada implementación en función de la estrategia empleada para la generación de subclaves (*round keys*): una variante en la que las subclaves se precalculan y almacenan antes del inicio del cifrado (llave precalculada), y otra en la que las subclaves se generan dinámicamente en cada ronda (llave al vuelo).

De forma análoga, cada fila de la tabla corresponde a una implementación distinta del cifrado, comenzando por la versión más básica. Esta implementación inicial se utiliza como referencia para evaluar el impacto de las distintas optimizaciones aplicadas en etapas posteriores, criterio que se mantiene de forma consistente en el resto de métricas software analizadas.

La implementación base presenta una latencia aproximada de 5.5 ms en ambas variantes de generación de clave, lo que constituye el punto de partida para la comparación de las mejoras obtenidas.

A partir de esta configuración inicial, se observan variaciones significativas en la latencia asociadas a las distintas optimizaciones aplicadas. En particular, todas las optimizaciones basadas en el uso de estructuras de búsqueda en tablas (*table lookup*) suponen una reducción clara respecto a la implementación original. En el caso más simple, correspondiente a la sustitución de la operación de permutación por una búsqueda directa en tabla, se obtiene una mejora considerable, reduciendo la latencia en más de un 50 % (52.25 % para la variante con clave en vuelo y 58.90 % para la variante con clave precalculada).

Esta mejora se incrementa de forma notable al combinar las operaciones de sustitución y permutación en una única etapa basada también en tablas. En este caso, la reducción de latencia alcanza aproximadamente un 70 % (70.40 % en la variante con clave precalculada y 69.93 % en la variante con clave en vuelo) cuando se emplea una tabla por cada nibble del estado (configuración denominada "tablas_sp_múltiples", correspondiente a ocho tablas en este trabajo).

Si en lugar de ello se utiliza una única tabla base a partir de la cual se derivan las restantes ("tabla_sp_unica"), las operaciones adicionales necesarias para dicha derivación introducen un ligero incremento en la latencia, aunque este efecto resulta marginal y no altera de forma significativa el comportamiento global del sistema.

Finalmente, la combinación de sustitución y permutación mediante tablas de 8 bits de entrada ("tabla_sp_8bit") proporciona la mejor observación en esta familia de optimizaciones, reduciendo la latencia en torno a un 80 % respecto a la implementación base (79.60 % para clave en vuelo y 78.67 % para clave precal-

culada), consolidándose como la estrategia más eficiente dentro de las variantes analizadas.

La última optimización analizada, correspondiente a la implementación mediante *bit-slicing*, no produce una reducción de la latencia, sino un incremento significativo de la misma. En concreto, la latencia aumenta un 160,66 % en la variante con clave al vuelo y un 171.14 % en la variante con clave precalculada respecto a la implementación básica.

Este comportamiento se debe a que la técnica *bit-slicing* requiere un mayor número de operaciones adicionales durante el proceso de cifrado. Por un lado, resulta necesario reorganizar los datos tanto al inicio como al final de la ejecución para transformar los bloques entre el formato convencional y la representación bitslice. Por otro lado, las operaciones fundamentales del esquema SPN se vuelven más complejas desde el punto de vista computacional, especialmente la etapa de sustitución, que pasa a implementarse mediante secuencias de operaciones booleanas elementales.

No obstante, debe tenerse en cuenta que esta implementación procesa simultáneamente 32 bloques de datos, por lo que el incremento observado en latencia individual no implica necesariamente un peor rendimiento global del sistema, aspecto que se analizará posteriormente mediante la métrica de throughput.

En términos generales, los resultados obtenidos siguen el comportamiento esperado para cada técnica de optimización. Aquellas variantes orientadas a reducir el número de operaciones mediante el uso de búsquedas en tablas presentan disminuciones significativas de la latencia, mientras que las implementaciones que introducen procesamiento adicional o transformaciones internas más complejas muestran incrementos en el tiempo de ejecución.

Implementación	**Latencia (ms) (llave precalculada)**	**Latencia (ms) (llave al vuelo)**
implementacion_base	5.44	5.72
lookup_permutacion	2.23	2.73
tablas_sp_multiples	1.61	1.72
tabla_sp_unica	1.65	1.80
tabla_sp_8bit	1.11	1.22
bitslicing	14.75	14.91

Tabla 5.1: Comparación de latencia media para implementaciones software

5.2.2. Rendimiento o Throughput

Los resultados de throughput obtenidos para las distintas implementaciones software pueden consultarse en la Tabla 5.2. En el caso de aquellas versiones que procesan un único bloque por ejecución —esto es, la implementación básica y las distintas variantes basadas en búsquedas en tablas— el rendimiento obtenido puede interpretarse prácticamente como el inverso de la latencia. Esto se debe a que, cuanto menor es el tiempo necesario para cifrar un bloque, mayor será el número de bloques que podrán procesarse en una misma unidad de tiempo.

De este modo, las mismas causas que explicaban la reducción de latencia observada en las optimizaciones basadas en table lookup justifican ahora el incremento de throughput respecto a la implementación original. La sustitución de operaciones lógicas y manipulaciones bit a bit por accesos directos a tablas reduce el número total de operaciones ejecutadas durante el cifrado, permitiendo procesar bloques a mayor velocidad.

En consecuencia, las implementaciones que presentaban menores latencias son también las que alcanzan mayores valores de rendimiento. En particular, la variante "tabla_sp_8bit", que combinaba sustitución y permutación mediante tablas de 8 bits, resulta nuevamente la implementación más eficiente dentro de esta familia de optimizaciones, alcanzando el mayor throughput entre todas las variantes basadas en búsquedas en memoria.

Por otro lado, el caso de la implementación mediante *bit-slicing* presenta un comportamiento diferente. Aunque esta variante incrementa notablemente la latencia individual del proceso de cifrado, dicha penalización queda compensada por su capacidad para procesar simultáneamente 32 bloques de datos en una única ejecución. Gracias a este paralelismo a nivel de bits, el throughput total obtenido resulta significativamente superior al del resto de implementaciones software analizadas, convirtiéndose en la variante con mayor rendimiento global.

Implementación	**Rendimiento (bloques/s) (llave previa)**	**Rendimiento (bloques/s) (llave al vuelo)**
implementacion_base	182.48	176.13
lookup_permutacion	443.40	367.64
tablas_sp_multiples	612.40	585.37
tabla_sp_unica	600.97	564.45
tabla_sp_8bit	884.01	818.86
bitslicing	2144.68	2136.15

Tabla 5.2: Comparación de rendimiento para implementaciones software

5.2.3. Consumo de memoria

En la Tabla 5.3 se muestran los valores de memoria obtenidos para las distintas implementaciones software. Concretamente, se distinguen tres magnitudes: la memoria estática necesaria para almacenar estructuras auxiliares, principalmente tablas; la memoria pico consumida dinámicamente durante la ejecución del algoritmo; y la memoria total, correspondiente a la suma de ambas contribuciones.

La primera diferencia significativa aparece entre las variantes con clave precalculada y aquellas que generan las subclaves al vuelo. Las implementaciones con llave precalculada presentan un mayor consumo de memoria pico debido a la necesidad de almacenar previamente todas las subclaves utilizadas durante el proceso de cifrado.

Por otra parte, entre la implementación básica y las distintas optimizaciones basadas en búsquedas en tablas apenas se observan diferencias en la memoria pico consumida durante la ejecución. Esto se debe a que la estructura dinámica del algoritmo y el flujo general de procesamiento permanecen prácticamente inalterados. Las diferencias relevantes aparecen, por tanto, en la memoria estática requerida por cada implementación.

En este sentido, el coste asociado a las optimizaciones *table lookup* resulta directamente proporcional al número y tamaño de las tablas auxiliares almacenadas. La implementación básica únicamente requiere almacenar la tabla de sustitución correspondiente a la S-box. En la variante "lookup_permutacion" se añaden además ocho tablas auxiliares para implementar la operación de permutación mediante búsquedas directas. De forma similar, la implementación "tablas_sp_múltiples" incorpora ocho tablas SP que combinan las operaciones de sustitución y permutación.

La variante "tabla_sp_unica" consigue reducir significativamente este coste al emplear una única tabla SP a partir de la cual se derivan las restantes mediante operaciones adicionales de desplazamiento. Gracias a ello, sus necesidades de memoria resultan comparables a las de la implementación básica.

Cabe destacar especialmente el caso de la implementación "tabla_sp_8bit", que era también la variante con mejor latencia y throughput dentro de las optimizaciones basadas en tablas. Sin embargo, esta mejora de rendimiento implica un incremento notable del consumo de memoria, ya que una tabla de 8 bits requiere almacenar (2^8) entradas, frente a las (2^4) entradas necesarias en las tablas de 4 bits empleadas en las variantes anteriores.

La implementación mediante bit-slicing presenta un comportamiento diferente al no requerir tablas auxiliares para realizar las operaciones de sustitución y permutación. Esto elimina prácticamente el consumo de memoria estática asociado a estructuras auxiliares, situándola en mejores condiciones que la mayoría de variantes basadas en búsquedas en tablas, salvo la implementación "tabla_sp_unica". No obstante, sí se observa un incremento en la memoria pico consumida durante la ejecución, debido a la necesidad de almacenar temporalmente la estructura

bit-slice, equivalente a una matriz que contiene simultáneamente los 32 bloques procesados en paralelo.

Implementación	Memoria estática (Bytes)	Estrategia: Llave Previa Memoria pico (Bytes)	Estrategia: Llave Previa Memoria total (Bytes)	Estrategia: Llave al Vuelo Memoria pico (Bytes)	Estrategia: Llave al Vuelo Memoria total (Bytes)
implementacion_base	314	868	1182	242	556
lookup_permutacion	3054	868	3922	242	3296
tablas_sp_multiples	2800	868	3668	242	3042
tabla_sp_unica	338	868	1206	242	580
tabla_sp_8bit	5114	868	5982	240	5354
bitslicing	0	2118	2118	1448	1448

Tabla 5.3: Comparación de consumo de memoria para implementaciones software

5.3. Resultados de las métricas hardware

5.3.1. Latencia

En la Tabla 5.4 se muestran los valores de latencia obtenidos para las distintas implementaciones hardware desarrolladas. La primera diferencia relevante respecto a las implementaciones software aparece en el orden de magnitud de las medidas obtenidas: mientras que en software las latencias se expresaban en milisegundos, en hardware estas pasan a medirse en microsegundos. Esto confirma la mejora de velocidad esperada al trasladar la ejecución del cifrado a lógica hardware dedicada sobre FPGA.

Las implementaciones analizadas corresponden a las distintas variantes descritas previamente en la metodología. En primer lugar, se incluye la implementación hardware básica, utilizada como referencia para evaluar el efecto de las distintas optimizaciones. De esta implementación básica se desarrolló además una variante con subclaves precalculadas, análoga a la versión software equivalente, donde las round keys se generan previamente y se almacenan en registros antes de comenzar el cifrado.

A continuación se incluyen las distintas variantes basadas en loop unrolling, en las que se desenrolla el bucle principal de rondas utilizando diferentes factores de desenrollado (2, 4, ..., completo). También se incorpora la implementación serializada, orientada a reducir el consumo de recursos mediante la reutilización de una única S-box.

Posteriormente se presentan las distintas implementaciones paralelas, capaces de replicar el cifrador múltiples veces (2, 4, 8, ...). Dentro de esta familia se distinguen tres aproximaciones: las implementaciones paralelas básicas ("Paralelo"), aquellas que extraen el key schedule mediante precálculo de subclaves

("Paralelo precálculo") y las que realizan esta separación mediante estructuras de tipo streaming ("Paralelo stream").

Finalmente, se muestran las implementaciones basadas en pipeline, que difieren entre sí en el número de rondas agrupadas dentro de cada etapa del pipeline (1, 2 o 4 rondas por etapa).

Analizando los resultados obtenidos, la primera observación relevante corresponde a las variantes que intentan separar el cálculo de subclaves del propio cifrador mediante precálculo. En estos casos se produce un incremento de la latencia respecto a la implementación básica, tanto en las versiones paralelas como en la implementación individual. Este comportamiento se debe a que el sistema pasa a dividirse en dos etapas diferenciadas —cálculo de subclaves y cifrado— incorporando además lógica adicional necesaria para controlar el acceso y distribución de dichas subclaves.

Especialmente interesantes resultan los resultados obtenidos en las variantes basadas en *loop unrolling*. Inicialmente podría esperarse una reducción de latencia al ejecutarse varias rondas simultáneamente. Sin embargo, los resultados muestran latencias prácticamente idénticas a la implementación básica e incluso ligeros incrementos en algunos casos. La causa principal reside en las dependencias existentes entre rondas consecutivas del algoritmo SPN: cada ronda necesita operar sobre el estado modificado por la ronda anterior, dificultando que la herramienta HLS pueda paralelizar correctamente el diseño. En consecuencia, el uso del pragma unroll no logra traducirse en una ejecución realmente concurrente y, en determinados casos, puede incluso introducir lógica adicional que penaliza el rendimiento, aspecto que también se reflejará posteriormente en las métricas de área.

La implementación serializada presenta igualmente un incremento de la latencia respecto a la implementación básica. Esto se debe a que la reutilización de una única S-box obliga a procesar secuencialmente los distintos nibbles del estado, aumentando el número total de iteraciones necesarias para completar cada ronda. No obstante, dado el reducido tamaño del bloque utilizado en este trabajo, el impacto total sobre la latencia resulta relativamente moderado.

En las implementaciones paralelas, la latencia se mantiene en valores similares a los de la implementación básica, ya que internamente cada línea de procesamiento sigue utilizando el mismo cifrador original, simplemente replicado múltiples veces. Aun así, puede observarse un ligero incremento de latencia conforme aumenta el número de cifradores paralelos. Este comportamiento se debe a que el diseño hardware resultante incorpora rutas de señal más largas y una distribución más compleja de señales compartidas, especialmente las asociadas al reloj y control global del sistema.

Dentro de esta familia de diseños paralelos, la variante basada en streaming para la distribución de subclaves consigue mejorar ligeramente los tiempos obtenidos. A diferencia de las versiones basadas en precálculo, en esta arquitectura el key schedule puede operar simultáneamente al cifrado, suministrando dinámica-

mente las subclaves mediante estructuras FIFO, lo que permite reducir parte de la sobrecarga temporal introducida por la gestión de claves.

Finalmente, las implementaciones pipeline presentan las mayores latencias individuales de todas las variantes analizadas, comportamiento coherente con la estructura interna de este tipo de diseños. En una arquitectura pipeline, cada bloque de datos debe atravesar múltiples etapas secuenciales separadas por registros intermedios, de forma que los resultados de cada etapa deben almacenarse y sincronizarse antes de continuar su procesamiento en la siguiente. Esto incrementa el tiempo necesario para completar el cifrado de un único bloque.

Además, los resultados muestran que reducir el número de etapas agrupando varias rondas dentro de una misma etapa no implica necesariamente una mejora de la latencia total. Aunque disminuye el número de registros y sincronizaciones necesarias, la complejidad combinacional de cada etapa aumenta considerablemente, incrementando el tiempo de propagación interno y compensando parcialmente la reducción del número de etapas.

Implementación	**Latencia Media (μs)**	**Implementación**	**Latencia Media (μs)**
Basico	41.23	Paralelo 4	42.00
Basico precalculo	43.08	Paralelo 8	41.64
Unroll 2	43.14	Paralelo 16	42.33
Unroll 4	42.41	Paralelo 32	43.33
Unroll 8	41.79	Paralelo precalculo 4	43.26
Unroll 10	41.40	Paralelo precalculo 8	42.42
Unroll 16	42.06	Paralelo Stream	41.34
Unroll 20	41.08	Pipeline 1	101.47
Unroll completo	41.90	Pipeline 2	105.86
Serial	42.09	Pipeline 4	107.44

Tabla 5.4: Comparación de latencia media en microsegundos (μs) para implementaciones hardware

5.3.2. Rendimiento o Throughput

Los resultados de throughput obtenidos para las distintas implementaciones hardware pueden consultarse en la Tabla 5.5. La primera diferencia destacable respecto a las implementaciones software aparece nuevamente en el orden de magnitud de los valores obtenidos. Mientras que en software el rendimiento se expresaba en bloques por segundo, en hardware los resultados se muestran en mi-

les de bloques por segundo para facilitar su lectura, reflejando así la considerable mejora de rendimiento proporcionada por la aceleración hardware sobre FPGA.

En aquellas implementaciones capaces de procesar únicamente un bloque de datos de forma simultánea —como la implementación básica, las variantes basadas en *loop unrolling* y la implementación serializada— el throughput apenas presenta variaciones significativas. Este comportamiento resulta coherente con los resultados observados previamente en la métrica de latencia, ya que en este tipo de arquitecturas el rendimiento se encuentra estrechamente relacionado con el tiempo necesario para completar el cifrado de un único bloque. Dado que dichas implementaciones mostraban latencias muy similares, el throughput obtenido permanece también en valores próximos.

Los resultados permiten apreciar con claridad las ventajas proporcionadas por las arquitecturas paralelas y pipeline. En el caso de las implementaciones paralelas, el rendimiento aumenta aproximadamente de forma proporcional al número de cifradores replicados en el diseño. De este modo, al duplicar el número de líneas de cifrado activas se obtiene un incremento cercano al doble en el throughput total, comportamiento esperado en este tipo de arquitecturas de paralelismo espacial.

Implementación	**Throughput (kbloques/s)**	**Implementación**	**Throughput (kbloques/s)**
Basico	21	Paralelo 4	92
Basico precalculo	20	Paralelo 8	184
Unroll 2	23	Paralelo 16	367
Unroll 4	23	Paralelo 32	736
Unroll 8	23	Paralelo precalculo 4	92
Unroll 10	23	Paralelo precalculo 8	184
Unroll 16	23	Paralelo Stream	91
Unroll 20	23	Pipeline 1	22 691
Unroll completo	24	Pipeline 2	23 011
Serial	21	Pipeline 4	22 838

Tabla 5.5: Comparación de throughput ajustado en miles de bloques por segundo (kbloques/s) para implementaciones hardware

No obstante, los mejores resultados corresponden a las implementaciones pipeline. Estas arquitecturas presentan, con diferencia, los mayores valores de throughput obtenidos en todo el conjunto de implementaciones hardware analizadas, llegando a triplicar ampliamente el rendimiento alcanzado por las versiones paralelas con un mayor número de cifradores replicados. Este comportamiento se

debe a que, una vez llenado el pipeline, el sistema es capaz de producir bloques cifrados de forma continua, permitiendo completar nuevos cifrados prácticamente en cada ciclo de reloj.

En consecuencia, mientras que las arquitecturas paralelas incrementan el rendimiento mediante replicación de hardware, las arquitecturas pipeline consiguen explotar simultáneamente el paralelismo temporal interno del propio algoritmo SPN, ofreciendo una utilización mucho más eficiente de los recursos hardware disponibles cuando el objetivo principal es maximizar el throughput global del sistema.

5.3.3. Consumo de Área

En la Tabla 5.6 se muestran los valores de área obtenidos a partir de los informes generados por Vivado tras la síntesis de cada diseño. Tal y como se explicó en la metodología, las métricas consideradas corresponden al consumo de distintos recursos hardware de la FPGA: LUT, LUTRAM, FF y BRAM.

Cabe destacar que los informes proporcionados por Vivado ofrecen información muy detallada sobre el uso de recursos, ofreciendo tanto el consumo global del diseño completo como el desglose asociado a cada una de los IPs integrados en el sistema. Este desglose permite diferenciar entre el área correspondiente a el IP desarrollado (el propio cifrador hardware) y la lógica adicional necesaria para su integración y comunicación con el procesador. En la tabla referenciada se presentan únicamente los valores correspondientes al diseño completo.

En la mayoría de implementaciones, este desglose interno no resulta especialmente relevante, ya que la estructura que rodea a los IPs desarrollados es prácticamente idéntica en todos los casos. Sin embargo, conviene diferenciar especialmente el caso de las implementaciones *pipeline*. Mientras que el resto de diseños se comunican con el procesador mediante interfaces AXI-Lite, las arquitecturas *pipeline* emplean interfaces AXI-Stream, requiriendo estructuras de comunicación significativamente más complejas. Como consecuencia, la lógica adicional ajena al propio cifrador aumenta considerablemente en estas implementaciones, añadiendo aproximadamente 2800 LUT y 3700 FF adicionales, frente a las cerca de 477 LUT y 620 FF presentes en el resto de diseños. Aun así, incluso considerando esta sobrecarga, las implementaciones *pipeline* continúan requiriendo menos área total que las arquitecturas paralelas con un elevado número de cifradores replicados.

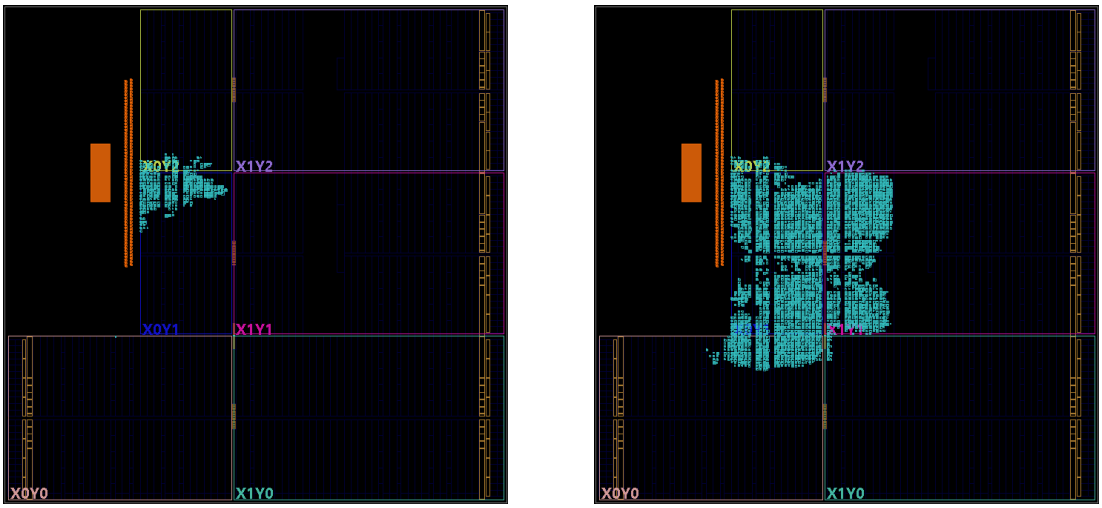
Teniendo en cuenta estas consideraciones sobre el diseño completo, también resulta importante señalar que buena parte de la LUTRAM reportada —habitualmente cuando aparecen valores iguales a 2— y las BRAM utilizadas en las implementaciones pipeline proceden realmente de los elementos auxiliares necesarios para la comunicación con el procesador, y no directamente del cifrador implementado.

Existen, no obstante, algunas excepciones relevantes. En las implementaciones “Paralelo stream”, las colas FIFO empleadas para comunicar el módulo key

schedule con los distintos cifradores requieren LUTRAM adicional. Del mismo modo, las arquitecturas pipeline también necesitan LUTRAM extra debido a los registros y buffers asociados a las etapas del pipeline. Otro caso particularmente destacable es el de la implementación unroll 20, donde la aplicación del pragma resulta especialmente desfavorable. En este diseño, la herramienta HLS genera una cantidad considerablemente mayor de lógica de control y elementos de almacenamiento respecto a otros factores de desenrollado, incrementando notablemente el consumo de recursos.

Analizando las distintas optimizaciones, se observa en primer lugar que las implementaciones basadas en *loop unrolling* incrementan progresivamente el área utilizada conforme aumenta el factor de desenrollado. Este comportamiento resulta esperable, ya que al paralelizar parcialmente las rondas del algoritmo es necesario sintetizar simultáneamente más lógica combinacional y estructuras de control adicionales.

Un comportamiento igualmente previsible aparece en las implementaciones paralelas. En este caso, el aumento del área es consecuencia directa de la replicación física completa del cifrador. Tal y como puede observarse en los resultados, el crecimiento del consumo de recursos es prácticamente lineal respecto al número de cifradores replicados. Este efecto puede apreciarse visualmente en la Figura 5.1, donde se comparan los mapas de utilización de recursos proporcionados por Vivado para la implementación básica y para la arquitectura paralela con 32 cifradores, observándose un incremento muy significativo de la región ocupada dentro de la FPGA.



(a) Utilización de la Implementación Básica

(b) Utilización de la Implementación Paralela 32

Figura 5.1: Comparativa de mapas de utilización

Continuando con los resultados esperables, también puede observarse un incre-

mento del área en aquellas implementaciones que realizan precálculo de subclaves. Este aumento se debe principalmente a la necesidad de almacenar las round keys generadas previamente, lo que implica un mayor uso de registros (Flip-Flops). Del mismo modo, resulta necesaria lógica adicional de control —reflejada principalmente en el aumento de LUT— encargada de gestionar el acceso y distribución de dichas subclaves durante el proceso de cifrado.

El caso más interesante corresponde a la implementación serializada. Esta arquitectura tenía como objetivo principal reducir el área respecto a la implementación hardware básica mediante la reutilización de una única S-box. Sin embargo, los resultados muestran que no solo no consigue reducir el consumo de recursos, sino que incluso requiere ligeramente más área.

Este incremento se debe principalmente a la lógica adicional necesaria para emular el comportamiento del registro de selección empleado durante el procesamiento secuencial de los nibbles. El diseño requiere numerosos registros auxiliares para almacenar temporalmente el estado interno, así como lógica adicional para seleccionar individualmente cada nibble, aplicar la sustitución correspondiente y reinsertar posteriormente el resultado dentro del bloque cifrado. Del mismo modo, las operaciones de permutación y el control secuencial completo del proceso introducen nuevas estructuras de control que incrementan el uso de LUT y FF.

Estos resultados evidencian una de las limitaciones prácticas del uso de HLS frente a descripciones hardware de más bajo nivel. Aunque teóricamente una arquitectura serializada puede reducir significativamente el área de implementación, la capacidad expresiva proporcionada por HLS no permite controlar con suficiente precisión determinados detalles microarquitectónicos del diseño generado. Como consecuencia, la herramienta introduce automáticamente estructuras auxiliares y lógica de control adicionales que terminan compensando e incluso superando parte de la reducción de recursos esperada teóricamente.

Implementación (Diseño Completo)	LUT	FF	LUTRAM	BRAM
Basico	723	900	2	0
Basico precalculo	794	1081	2	0,5
Unroll 2	812	940	2	0
Unroll 4	854	1003	2	0
Unroll 8	994	1195	0	0
Unroll 10	1080	1302	2	0
Unroll 16	1272	1580	2	0
Unroll 20	1809	1900	81	0
Unroll 31	1735	2109	2	0
Serial	849	1030	2	0
Paralelo 4	1456	1739	2	0
Paralelo 8	2023	2678	2	0
Paralelo 16	3351	4553	2	0
Paralelo 32	6100	8300	2	0
Paralelo precalculo 4	2647	2741	2	0
Paralelo precalculo 8	4644	3680	2	0
Paralelo Stream	1877	2278	194	0
Pipeline 1	4190	6138	158	2
Pipeline 2	4185	5578	158	2
Pipeline 4	4215	5837	158	2

Tabla 5.6: Comparación de consumo de área del diseño completo en la FPGA

5.3.4. Consumo de potencia

En la Tabla 5.7 se muestran los valores correspondientes al consumo de potencia dinámica estimado por Vivado para cada una de las implementaciones desarrolladas sobre la lógica programable de la FPGA. Además de estos valores, los informes generados por la herramienta incluyen otros dos consumos prácticamente invariantes entre diseños: por un lado, el asociado al procesador ARM del sistema, con un valor constante aproximado de 1.260 W, y por otro la potencia estática del dispositivo, situada entre 0.134 y 0.135 W en todas las implementaciones analizadas.

Los resultados obtenidos muestran que esta métrica no actúa como un gran factor diferenciador entre las distintas variantes implementadas, ya que la variabilidad observada en el consumo dinámico resulta relativamente reducida. En términos generales, puede apreciarse una relación directa entre el área ocupada y la potencia consumida: aquellas arquitecturas que requieren una mayor cantidad de recursos hardware —como las implementaciones paralelas o *pipeline*—

son también las que presentan un mayor consumo energético.

Este comportamiento resulta coherente, dado que un mayor número de elementos lógicos activos implica un incremento de las conmutaciones internas y, por tanto, del consumo dinámico asociado a la lógica programable. No obstante, las diferencias absolutas entre implementaciones continúan siendo moderadas, especialmente si se comparan con el consumo fijo introducido por el propio sistema embebido y el procesador ARM de la plataforma.

Implementación	Clocks (W)	Signals (W)	Logic (W)	BRAM (W)	Total PL (W)
Basico	0.004	0.003	0.003	0.000	**0.010**
Basico precalculo	0.005	0.003	0.003	0.001	**0.012**
Unroll 2	0.004	0.003	0.004	0.000	**0.011**
Unroll 4	0.005	0.004	0.004	0.000	**0.013**
Unroll 8	0.005	0.005	0.006	0.000	**0.016**
Unroll 10	0.004	0.004	0.005	0.000	**0.013**
Unroll 16	0.005	0.004	0.005	0.000	**0.014**
Unroll 20	0.009	0.013	0.017	0.000	**0.039**
Unroll 31	0.005	0.002	0.002	0.000	**0.009**
Serial	0.004	0.002	0.002	0.000	**0.008**
Paralelo 4	0.006	0.006	0.007	0.000	**0.019**
Paralelo 8	0.007	0.010	0.011	0.000	**0.028**
Paralelo 16	0.011	0.020	0.019	0.000	**0.050**
Paralelo 32	0.018	0.037	0.037	0.000	**0.092**
Paralelo precalculo 4	0.008	0.012	0.011	0.000	**0.031**
Paralelo precalculo 8	0.015	0.025	0.021	0.000	**0.061**
Paralelo Stream	0.010	0.007	0.007	0.000	**0.024**
Pipeline 1	0.020	0.010	0.012	0.000	**0.042**
Pipeline 2	0.019	0.013	0.016	0.000	**0.048**
Pipeline 4	0.019	0.013	0.017	0.000	**0.049**

Tabla 5.7: Desglose de potencia dinámica en la lógica programable (PL) para las diferentes implementaciones hardware

5.4. Métricas para el PRESENT-80

Tal y como se explicó en la metodología, la adaptación de las implementaciones desarrolladas al caso práctico de PRESENT-80 únicamente requiere ajustar determinados parámetros estructurales del SPN original. Los resultados obtenidos para las implementaciones básicas de PRESENT-80 se recogen en las Tablas 5.8 y 5.9, correspondientes respectivamente a las versiones software con cálculo de subclaves al vuelo y hardware. La comparación con las implementaciones básicas del SPN teórico permite analizar el impacto de dicha adaptación sobre las distintas métricas.

Rendimiento (bloques/s)		**Latencia (ms)**
85,39		11,46
	Memoria	
Memoria Pico (B)	**Estruct. Auxiliar (B)**	**Memoria Total (B)**
1000	314	**1314**

Tabla 5.8: Métricas finales de la implementación PRESENT en SW.

En el caso del software, la latencia aproximadamente se duplica respecto al SPN original. Este incremento se debe principalmente al aumento del tamaño de bloque de 32 a 64 bits, lo que implica un mayor número de nibbles y, por tanto, el doble de operaciones de sustitución mediante S-box. Dado que estas operaciones se realizan de forma iterativa, el incremento se traduce directamente en un mayor tiempo de ejecución. De forma análoga, la permutación también requiere procesar más bits, contribuyendo al aumento global de la latencia.

Por el contrario, en hardware la variación es reducida. El paralelismo de la FPGA permite implementar las S-box necesarias de forma concurrente, de modo que el aumento del número de operaciones apenas afecta al tiempo de ejecución, manteniéndose latencias similares al caso del SPN original.

Dado que en estas implementaciones el throughput está estrechamente ligado a la latencia, el comportamiento es equivalente: en hardware apenas varía, mientras que en software se reduce aproximadamente a la mitad.

En cuanto a memoria, se observa un ligero incremento en software debido al mayor tamaño de bloques y claves, que requieren estructuras de almacenamiento más amplias. De forma consistente, en hardware también aumenta el consumo de recursos, al ser necesarios más S-box, registros de mayor tamaño y lógica adicional para la permutación y el key schedule.

Finalmente, el consumo de potencia estimado por Vivado aumenta ligeramente, en coherencia con el mayor uso de área y recursos lógicos del diseño.

Rendimiento (kbloques/s)		**Latencia (μs)**	
22,48		42,79	
Consumo de área (Diseño Completo)			
LUT	**FF**	**LUTRAM**	**BRAM**
916	1257	2	1
Consumo Dinámico PL (W)			
Clocks	**Signals**	**Logic**	**BRAM**	**Total PL**
0,005	0,003	0,003	0,002	**0,013**

Tabla 5.9: Métricas finales de la implementación PRESENT en HW.

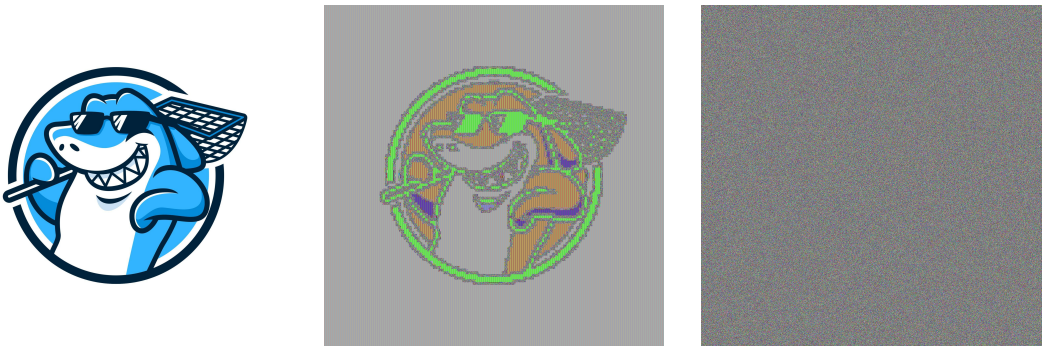
5.5. Modo de cifrado CTR

Tal y como se mencionó en la metodología, se implementó una versión paralela del cifrador empleando el modo de operación CTR (Counter Mode). Concretamente, se tomó como base la implementación paralela con 32 cifradores, correspondiente a la versión de mayor rendimiento desarrollada durante el trabajo. No se discutirán en detalle las métricas obtenidas para esta variante, ya que apenas presentan diferencias respecto a la implementación paralela equivalente en modo ECB. Esta similitud resulta esperable, dado que las modificaciones necesarias para implementar CTR son mínimas: únicamente se requiere incorporar un contador, lógica para incrementarlo tras cada bloque procesado y una operación XOR adicional para combinar el bloque de texto plano con el contador cifrado.

Empleando una imagen de prueba sencilla, puede ejemplificarse visualmente la diferencia entre los modos de cifrado ECB y CTR. La imagen utilizada se caracteriza por contener amplias regiones de colores uniformes, lo que implica la aparición de múltiples bloques idénticos durante el proceso de cifrado. Esta situación resulta especialmente vulnerable frente a ataques por análisis de patrones, ya que en el modo ECB bloques de entrada iguales producen bloques cifrados también iguales.

Puede observarse en las figuras 5.2 la diferencia visual entre ambos modos de operación. En el caso de ECB, continúan apreciándose contornos y patrones de la imagen original, evidenciando la vulnerabilidad frente a ataques por análisis de patrones. Por el contrario, en el modo CTR la imagen cifrada presenta una apariencia similar a un mapa de ruido aleatorio, ocultando correctamente la estructura visual de la imagen original.

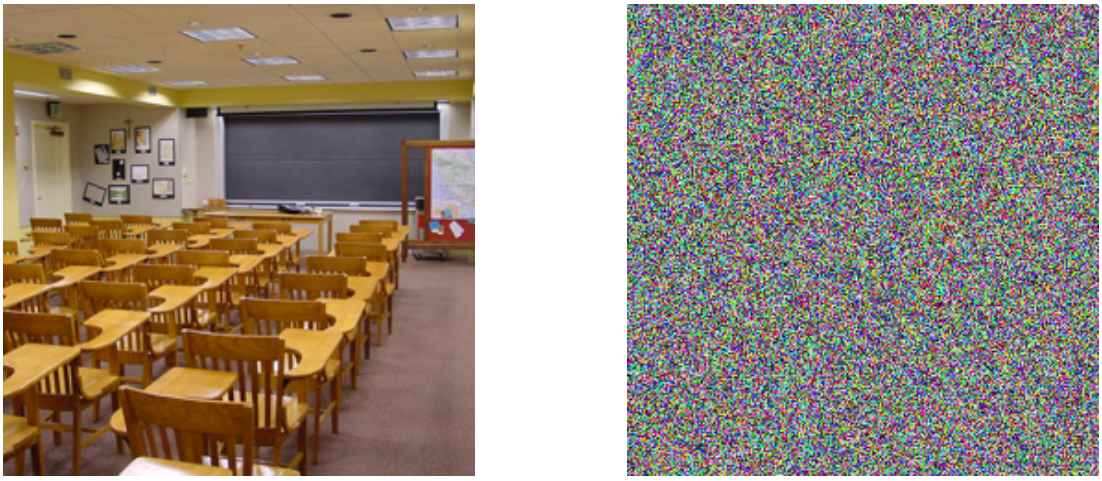
Este mismo mecanismo de cifrado de imagenes con CTR se ha empleado para realizar una prueba práctica de cifrado de imágenes pertenecientes a un conjunto de datos de tipo CCTV [13] presente en las figuras 5.3. El objetivo de esta prueba



(a) Original (b) Cifrada con ECB (c) Cifrada con CTR

Figura 5.2: Comparación entre ECB y CTR

es ilustrar un escenario de aplicación realista para este tipo de arquitecturas, como puede ser la protección de información visual sensible capturada por sistemas de videovigilancia o dispositivos IoT antes de su transmisión.



(a) Original (b) Cifrada con CTR

Figura 5.3: Caso práctico de cifrado de imagenes CCTV [13]

Debe señalarse que, con las implementaciones desarrolladas en este trabajo, el cifrado completo de las imágenes de prueba requiere todavía varios segundos de ejecución. Esto impide alcanzar velocidades compatibles con aplicaciones de vídeo en tiempo real, como el estándar de 24 imágenes por segundo. Para alcanzar dicho objetivo sería necesario desarrollar arquitecturas de mayor rendimiento, por ejemplo incrementando el grado de paralelización del cifrador o empleando plataformas FPGA con mayores capacidades de procesamiento.

Capítulo 6

Conclusiones y posibles ampliaciones

En términos generales, se han cumplido los objetivos propuestos y se ha verificado la hipótesis principal del trabajo: la implementación hardware de algoritmos criptográficos de tipo SPN sobre FPGA permite obtener mejoras significativas en términos de latencia y rendimiento frente a implementaciones software. Tal como se observa en los resultados obtenidos, las versiones hardware presentan sistemáticamente mejores valores de latencia y throughput.

Asimismo, los resultados permiten validar la hipótesis del trabajo relativa al impacto de las optimizaciones aplicadas. Se ha comprobado que las distintas técnicas afectan de manera diferente al equilibrio entre las métricas consideradas. En el caso del software, las optimizaciones basadas en búsqueda en tablas reducen de forma significativa la latencia a costa de un mayor consumo de memoria, mientras que el bit-slicing la incrementa para lograr un aumento notable del rendimiento global al procesar múltiples bloques simultáneamente. En el ámbito hardware, la paralelización y el uso de pipelines incrementan el throughput de forma considerable, aunque con compromisos claros: el primero requiere un mayor consumo de área al replicar unidades de procesamiento, y el segundo introduce una mayor latencia inicial debida al llenado de las etapas del pipeline. En conjunto, estos comportamientos responden a lo esperado teóricamente y a lo expuesto en la metodología.

Sin embargo, también se han identificado casos en los que los resultados no han sido los esperados. La técnica de loop unrolling no ha alcanzado el impacto previsto debido a la presencia de dependencias entre rondas del cifrado, mientras que la serialización no ha logrado una reducción significativa del área del diseño. Estas observaciones, junto con la implementación del modo de operación CTR sobre arquitecturas paralelas, permiten extraer una serie de lecciones adicionales.

En primer lugar, HLS constituye una herramienta muy útil para agilizar el desarrollo de un amplio conjunto de IPs, pero no es infalible. Su menor flexibilidad puede dificultar o incluso impedir la implementación de diseños más sensibles o

que requieran un mayor control a bajo nivel, como se ha evidenciado en el caso de la serialización. aunque las métricas estudiadas permiten evaluar el impacto de distintas optimizaciones sobre algoritmos SPN, no debe olvidarse que en criptografía la seguridad es un requisito fundamental. La elección de optimizaciones no es útil si compromete la protección de la información. Un ejemplo ilustrativo es el uso de modos de cifrado en la implementación paralelizada aplicada al cifrado de imágenes: si se cifran bloques de forma independiente (ECB), se exponen patrones estructurales, por lo que en escenarios reales deben emplearse modos seguros como CTR o equivalentes.

Los resultados obtenidos en este trabajo deben interpretarse teniendo en cuenta una serie de limitaciones inherentes tanto a la metodología empleada como a las herramientas de desarrollo utilizadas. En primer lugar, las métricas de consumo de potencia y recursos hardware se basan en estimaciones proporcionadas por las herramientas de síntesis de Vivado, y no en mediciones físicas directas sobre la plataforma, lo que implica que dichos valores deben entenderse como referencias comparativas entre diseños más que como medidas absolutas. Del mismo modo, las evaluaciones software se ven condicionadas por el entorno de ejecución basado en Linux y Python dentro de la plataforma PYNQ, lo que introduce cierta variabilidad en las medidas a pesar de la aplicación de técnicas de mitigación como warm-up y promediado de ejecuciones.

El trabajo realizado puede continuar desarrollándose fuera del alcance de esta memoria. Actualmente se dispone de múltiples bloques IP modulares que pueden integrarse en diseños en Vivado. Por ejemplo, es posible partir de un sistema completo basado en PYNQ-Z2 e integrar el módulo cifrador IP más adecuado según la aplicación. Esto permitiría procesar información procedente de sensores u otros dispositivos conectados a la placa, que posteriormente sería cifrada mediante las implementaciones desarrolladas, siempre que la capacidad computacional lo permita (por ejemplo, en el caso de vídeo podría ser necesario recurrir a hardware de mayor potencia). En esta misma línea, también podría explorarse una solución híbrida en la que la PL se encargue de la adquisición de datos y el procesador realice el cifrado mediante una implementación software.

Un caso de estudio adicional de interés sería comparar, al menos para la técnica de serialización, el rendimiento alcanzable mediante un diseño en HDL manual frente al obtenido mediante las herramientas actuales de HLS.

Otra posible línea de trabajo futuro consistiría en profundizar en la combinación de distintas optimizaciones hardware. A lo largo del trabajo se ha comprobado que técnicas como la paralelización o el pipeline permiten mejorar significativamente el rendimiento de forma independiente, por lo que resultaría interesante explorar arquitecturas híbridas más complejas que combinen simultáneamente varias de estas estrategias. No obstante, este tipo de diseños implican normalmente compromisos adicionales, especialmente en términos de consumo de área y complejidad del sistema, por lo que sería necesario estudiar cuidadosamente el equilibrio alcanzado entre rendimiento y utilización de recursos hardware.

Apéndice A

Manuais técnicos

A.1. Material proporcionado

El material necesario para reproducir los experimentos realizados en este trabajo se encuentra disponible en el directorio de Google Drive accesible desde el siguiente enlace: `https://drive.google.com/drive/folders/16CfkqXVE6j6TAtpXlXvb5h_PPABVwVTJ?usp=sharing`

El contenido del directorio se organiza de la siguiente forma:

- **tfg_code.ipynb**: documento interactivo de Jupyter Notebook que contiene el código necesario para la ejecución de las distintas implementaciones desarrolladas, tanto software como hardware, así como las funciones empleadas para la obtención de métricas de latencia, throughput y consumo de memoria.

- **/vitis**: directorio que contiene íntegramente los proyectos desarrollados en Vitis HLS necesarios para la generación de las distintas IPs hardware implementadas.

- **/vivado**: directorio que contiene los proyectos Vivado empleados para la integración de las IPs en el sistema hardware completo y la posterior generación de los bitstreams correspondientes.

- **overlays.zip**: conjunto de directorios que contiene los bitstreams y archivos auxiliares necesarios para importar los distintos overlays al sistema de archivos de PYNQ y poder cargar las implementaciones hardware desde el entorno Jupyter Notebook.

El objetivo de este material es permitir tanto la reconstrucción completa del flujo de diseño hardware/software desarrollado durante el trabajo como la reproducción directa de los experimentos y métricas presentadas en la memoria.

A.2. Preparación del entorno PYNQ

La configuración del entorno PYNQ-Z2 se realiza siguiendo la guía oficial disponible en la documentación de pynq:

`https://pynq.readthedocs.io/en/latest/getting_started/pynq_z2_setup.html`

A.2.1. Grabación de la imagen en la tarjeta microSD

La placa PYNQ-Z2 utiliza un sistema operativo Linux preconfigurado basado en PYNQ, distribuido como una imagen arrancable para tarjetas microSD. La preparación de la tarjeta se realiza siguiendo los pasos:

1. Descarga de la imagen oficial correspondiente a la placa PYNQ-Z2 desde la página oficial de PYNQ.
2. Descompresión del archivo hasta obtener la imagen con extensión `.img`.
3. Inserción de una tarjeta microSD (mínimo recomendado: 8 GB) en el ordenador.
4. Grabación de la imagen mediante una herramienta como Win32 Disk Imager o equivalente.
5. Expulsión segura de la tarjeta e inserción en la ranura microSD de la placa.

Según la documentación oficial, es necesario configurar el jumper de arranque en modo SD para permitir el inicio del sistema desde la tarjeta microSD.

A.2.2. Primer arranque de la placa

Una vez preparada la tarjeta, el primer arranque de la PYNQ-Z2 se realiza siguiendo la secuencia:

1. Configurar el jumper de arranque en modo SD (si no se ha hecho previamente).
2. Insertar la tarjeta microSD en la placa.
3. Configurar el jumper de alimentación en modo USB.
4. Conectar el cable micro-USB al puerto PROG/UART.
5. Conectar el cable Ethernet.
6. Encender la placa mediante el interruptor de alimentación.

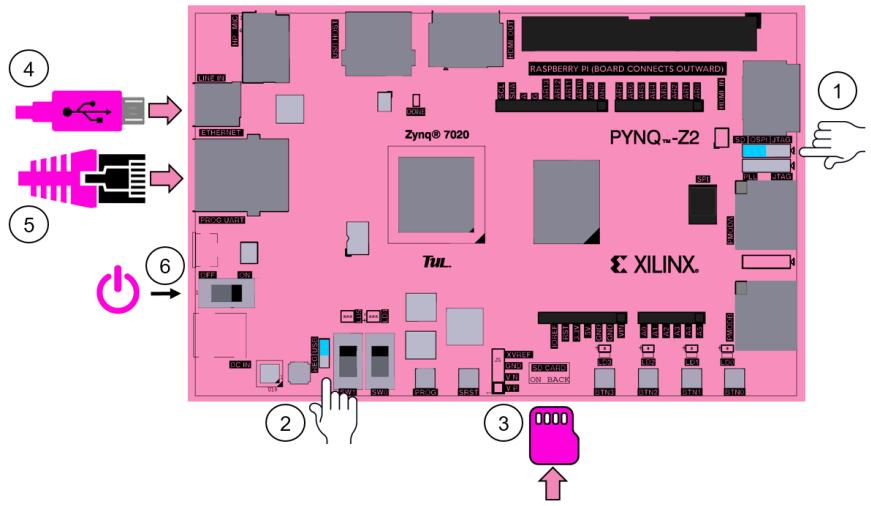


Figura A.1: Guía de arranque de pynq-z2

Durante el arranque, el LED rojo de alimentación permanece encendido inicialmente y, tras unos segundos, el LED DONE indica la correcta configuración del dispositivo FPGA. Posteriormente, la actividad de varios LEDs indica la finalización del arranque del sistema operativo Linux.

A.2.3. Acceso al entorno PYNQ

La placa utiliza por defecto la dirección IP `192.168.2.99`. Por tanto, es necesario configurar el ordenador en la misma subred (`192.168.2.x`) en la interfaz Ethernet utilizada.

Una vez establecida la conexión, es posible acceder a la interfaz web de PYNQ mediante cualquiera de las siguientes direcciones:

`http://192.168.2.99:9090/` o `http://pynq:9090/`

Las credenciales por defecto son:

- Usuario: `xilinx`
- Contraseña: `xilinx`

Tras la autenticación, se accede al entorno Jupyter Notebook integrado en PYNQ, donde se encuentran los notebooks de ejemplo y las herramientas de desarrollo.

A.2.4. Acceso al sistema de archivos

El sistema de archivos de la placa puede accederse desde el explorador del sistema operativo mediante conexión de red, utilizando la ruta:

`\\pynq`

Las credenciales de acceso son las mismas que para el entorno web (`xilinx/xilinx`).

A.3. Preparación del entorno Vivado y Vitis

A.3.1. Requisitos y versión utilizada

Para el desarrollo del proyecto se ha utilizado el entorno de diseño de AMD/-Xilinx, compuesto por Vivado Design Suite y Vitis HLS, disponible en:

`https://www.xilinx.com/support/download.html/content/xilinx/en/downloadNav/vivado-design-tools.html`

Se ha trabajado con la versión 2025.x del conjunto de herramientas, instalada mediante el Unified Installer, que integra en un único paquete Vivado, Vitis y Vitis HLS.

En cuanto a los requisitos del sistema, se recomienda disponer de:

- Sistema operativo de 64 bits (Windows o Linux)
- Memoria RAM mínima de 16 GB (recomendado 32 GB para diseños complejos)
- Espacio en disco superior a 100 GB para la instalación completa

A.3.2. Vitis HLS: generación e importación de IP

El desarrollo de los bloques hardware se realiza en Vitis HLS, donde la funcionalidad del sistema se describe mediante código en alto nivel (C/C++). A partir de este diseño, se genera un componente hardware sintetizable que posteriormente puede ser integrado en Vivado.

El flujo seguido en esta etapa es el siguiente:

1. Creación de un nuevo proyecto o componente HLS.
2. Selección del dispositivo objetivo correspondiente a la FPGA utilizada (Zynq-7000 en el caso de la PYNQ-Z2).
3. Implementación del diseño funcional en lenguaje C/C++.

4. Aplicación de directivas de síntesis para definir interfaces y optimizaciones del diseño.

5. Ejecución del proceso de síntesis para obtener la implementación hardware.

6. Empaquetado del resultado como IP compatible con Vivado.

Como resultado, se obtiene un bloque IP reutilizable que contiene la lógica hardware sintetizada junto con los archivos necesarios para su integración en sistemas de mayor complejidad.

A.3.3. Vivado: integración del sistema y generación del bitstream

La integración del sistema completo se realiza en Vivado Design Suite, donde se construye la arquitectura hardware final mediante un diseño basado en bloques.

El flujo de trabajo en esta fase es el siguiente:

1. Creación de un proyecto RTL y selección de la placa o dispositivo objetivo.

2. Diseño del sistema mediante el entorno Block Design, incorporando el procesador y los periféricos necesarios.

3. Importación del IP generado previamente desde Vitis HLS.

4. Interconexión de los bloques y configuración del sistema mediante herramientas de automatización del entorno.

5. Ejecución del proceso de síntesis e implementación.

6. Generación del bitstream final para la configuración de la FPGA.

Como resultado se obtienen los archivos necesarios para la programación de la placa, incluyendo el bitstream y los descriptores asociados al diseño hardware.

A.4. Reproducción de las métricas

El experimento completo puede reproducirse utilizando el material proporcionado. Para la obtención de las métricas de latencia, throughput y consumo de memoria, es necesario acceder al entorno Jupyter Notebook de PYNQ desde el navegador e importar el archivo `tfg_codigo.ipynb`.

Una vez importado el notebook, este puede ejecutarse prácticamente en su totalidad. Sin embargo, antes de ejecutar las celdas correspondientes a las implementaciones hardware, es necesario importar los *overlays*. Para ello, se debe acceder al sistema de ficheros de PYNQ y copiar el contenido del archivo

`overlays.zip` en el directorio correspondiente. En algunos casos puede ser necesario ajustar en el código las rutas utilizadas para la carga de los overlays.

Tras esta configuración inicial, la ejecución completa del notebook permite obtener todas las métricas experimentales. Los resultados se muestran en la sección *Métricas*, en formato de tablas generadas automáticamente.

Para la obtención de las métricas de consumo de área y potencia es necesario utilizar Vivado Design Suite. Una vez abierto el proyecto correspondiente, desde el panel *Flow Navigator* se selecciona la opción *Open Implemented Design*, lo que permite acceder a las estimaciones del diseño implementado.

En esta vista se encuentran los principales reportes utilizados en el análisis:

- **Report Power**: proporciona las estimaciones de consumo de potencia del diseño.
- **Report Utilization**: muestra el uso de recursos hardware (LUTs, FFs, BRAM, entre otros).
- **Schematic**: permite visualizar la estructura interna del diseño y los recursos de lógica programable utilizados.

Bibliografía

[1] H. M. Heys, "A Tutorial on the Implementation of Block Ciphers: Software and Hardware Applications", *IACR Cryptology ePrint Archive*, Report 2020/1545, 2020.

[2] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin and C. Vikkelsoe, "PRESENT: An Ultra-Lightweight Block Cipher", *Cryptographic Hardware and Embedded Systems – CHES 2007*, Lecture Notes in Computer Science, vol. 4727, pp. 450–466, 2007. doi:10.1007/978-3-540-74735-2_31.

[3] C. E. Shannon, "Communication Theory of Secrecy Systems", *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[4] National Institute of Standards and Technology, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions", *NIST Special Publication 800-232*, 2025. doi:10.6028/NIST.SP.800-232.

[5] J. G. Pandey, "Hardware architectures for PRESENT block cipher and their FPGA implementations", *IET Circuits, Devices & Systems*, vol. 13, no. 7, pp. 958-966, 2019. doi:10.1049/iet-cds.2018.5273.

[6] Michael D. Zwagerman, "High Level Synthesis, a Use Case Comparison with Hardware Description Language", *Master's Thesis, Grand Valley State University*, 2015.

[7] Hanchen Ye and Cong Hao and Jianyi Cheng and Hyunmin Jeong and Jack Huang and Stephen Neuendorffer and Deming Chen, "ScaleHLS: Scalable High-Level Synthesis through MLIR", *arXiv preprint arXiv:2107.11673*, 2021.

[8] Stefan Abi-Karam and Yuqi He and Rishov Sarkar and Lakshmi Sathidevi and Zihang Qiao and Cong Hao, "GenGNN: A Generic FPGA Framework for Graph Neural Network Acceleration", *arXiv preprint arXiv:2201.08475*, 2022.

[9] Michelangelo Barocci and Vittorio Fra and Enrico Macii and Gianvito Urgese, "Heterogeneous SoC Integrating an Open-Source Recurrent SNN Accelerator for Neuromorphic Edge Computing on FPGA", *arXiv preprint arXiv:2605.12217*, 2026.

[10] T. M. Kumar, K. S. Reddy, S. Rinaldi, B. D. Parameshachari e K. Arunachalam, "A Low Area High Speed FPGA Implementation of AES Architecture for Cryptography Application", *Electronics*, vol. 10, no. 16, art. 2023, 2021. doi:10.3390/electronics10162023.

[11] D. A. Osvik, J. W. Bos, D. Stefan e D. Canright, "Fast Software AES Encryption", *Fast Software Encryption (FSE)*, 2009.

[12] L. Daoud, F. Hussein e N. Rafla, "High-Level Synthesis Optimization of AES-128/192/256 Encryption Algorithms", *International Journal of Computers and Their Applications*, vol. 26, pp. 129–136, 2019.

[13] C. Werner, "Human Detection Dataset", *Kaggle Datasets*, 2021.

[14] Advanced Micro Devices (AMD), "PYNQ-Z2", *AMD University Program*, 2024.